ANGULAR 21 · EJERCICIO 02



Signals y directivas en Angular

Guía paso a paso: un CRUD de productos con **signal**, **computed**, **effect** y el control de flujo **@if** / **@for**

`signal()` `computed()` `effect()` `@for · @if` `Inmutabilidad` `localStorage`

10 etiquetas · 27 pasos

Octubre de 2026

Antes de empezar

Este documento recoge, en orden, los pasos seguidos para construir este ejemplo. El objetivo es trabajar con **signals** y con las directivas de control de flujo de Angular, mostrando una lista de productos en forma de cards y construyendo poco a poco un **CRUD** (Create, Read, Update, Delete) sobre ese listado.

Cada parte corresponde a un commit con su etiqueta: al hacer checkout de una etiqueta, el código y esta guía muestran el estado de ese paso.

Las etiquetas del proyecto

Cada etiqueta de git es un punto de control: con `git checkout <etiqueta>` el código y esta guía quedan en el estado de ese paso.

`01_listado` — pasos 1–6

leer y mostrar los productos con un signal y `@for` (la *R* de CRUD).

`02_boton_nuevo` — pasos 7–8

botón **Nuevo** que muestra, con `@if`, el `div` donde se dará de alta un producto, y botón **Cancelar** que lo oculta (primer paso de la *C* de CRUD).

`03_alta_producto` — pasos 9–13

campos del producto enlazados con sus signals mediante `[value]` y `(input)`, y botón **Guardar** que añade el producto a la lista de forma inmutable (la *C* de CRUD completa).

`04_borrado_producto` — pasos 14–15

botón **Borrar** en cada card que elimina el producto de forma inmutable con `filter` (la *D* de CRUD).

`05_boton_editar` — pasos 16–17

botón **Editar** en cada card que abre el `div` en modo edición con los datos del producto (primer paso de la *U* de CRUD).

`06_guardar_edicion` — pasos 18–19

Guardar crea o actualiza según el modo; la edición sustituye el producto con `map` y el spread de objetos (la *U* de CRUD completa).

`07_computed` — pasos 20–21

signal **calculado** con `computed()`, `totalProducts`, que muestra junto al título el número de productos y se actualiza solo al crear o borrar.

`08_filtro_titulo` — pasos 22–23

filtro por título con un signal calculado, `filteredProducts`, que depende de dos signals (`products` y `searchText`), y un `computed` que depende de otro `computed` para contar los productos filtrados.

`09_filtro_categoria` — pasos 24–25

filtro por categoría con un `<select>` y el signal `searchCategory`, que se combina con el filtro por título dentro del mismo `filteredProducts`.

`10_effect_localstorage` — pasos 26–27

persistencia de los productos en `localStorage` con un `effect()`, que guarda la lista cada vez que cambia el signal `products`.

Índice

`01_listado`

1 Punto de partida: proyecto, modelos y mock

2 Crear el componente `Products`

3 Crear el signal `products`

4 La directiva `@for`

5 Estilos: el HTML sin clases

6 Mostrar el componente en `App`

`02_boton_nuevo`

7 Crear productos: planteamiento

8 Mostrar y ocultar el div: la directiva `@if`

`03_alta_producto`

9 Los campos del producto: planteamiento

10 Un signal por cada campo

11 Enlazar los campos con los signals: `[value]` y `(input)`

12 Modificar un signal de tipo array: la inmutabilidad

13 Guardar el producto y cerrar el div

`04_borrado_producto`

14 Borrar productos: planteamiento

15 El botón Borrar y el método `deleteProduct()`

`05_boton_editar`

16 Editar productos: planteamiento

17 El modo del div y el producto seleccionado

`06_guardar_edicion`

18 Guardar la edición: planteamiento

19 `save()`, `updateProduct()` y la inmutabilidad en objetos

`07_computed`

20 Valores derivados: planteamiento

21 El signal calculado `totalProducts`: `computed()`

`08_filtro_titulo`

22 Filtrar productos: planteamiento

23 El filtro: `searchText`, `filteredProducts` y `totalFilteredProducts`

`09_filtro_categoria`

24 Filtrar por categoría: planteamiento

25 El filtro por categoría: `searchCategory` y `filteredProducts`

`10_effect_localstorage`

26 Persistencia de datos: planteamiento

27 Guardar los productos en `localStorage` con `effect()`

ETIQUETA DE GIT · PASOS 1–6

01_listado

leer y mostrar los productos con un signal y `@for` (la *R* de CRUD).

```
$ git checkout 01_listado
```

1 Punto de partida: proyecto, modelos y mock

El proyecto se creó con Angular CLI (Angular 21, sin SSR y con CSS):

```bash
ng new ejemplo_02 --style css --ssr false
```

Desde Angular 20 los proyectos nuevos son **zoneless** (sin zone.js) por defecto. Esto es importante para los signals, como se verá en el paso 3.

Dentro de `src/app` se creó la organización habitual en proyectos Angular:

```
src/app/
  core/
    services/        # servicios singleton (providedIn: 'root')
    guards/          # guards de rutas
    interceptors/    # interceptores HTTP
    models/          # interfaces y tipos compartidos
    mocks/           # datos de ejemplo estáticos
  shared/
    components/      # componentes reutilizables
    directives/      # directivas reutilizables
    pipes/           # pipes reutilizables
  features/
    products/        # funcionalidad "productos"
```

Cada carpeta vacía lleva un `.gitkeep` para que Git la guarde. Las imágenes de los productos están en `public/images` (de `1.png` a `20.png`) y se sirven desde la raíz: `/images/1.png`.

Los datos que se van a mostrar están tipados en `src/app/core/models`. La categoría es un enum con los 4 valores posibles:

```ts
// core/models/category.model.ts
export enum Category {
  MenClothing = "men's clothing",
  WomenClothing = "women's clothing",
  Jewelery = 'jewelery',
  Electronics = 'electronics'
}
```

```ts
// core/models/rating.model.ts
export interface Rating {
  rate: number;
  count: number;
}
```

```ts
// core/models/product.model.ts
export interface Product {
  id: number;
  title: string;
  price: number;
  description: string;
  category: Category;
  image: string;
  rating: Rating;
}
```

Y el array con los 20 productos está en `src/app/core/mocks/products.mock.ts`:

```ts
export const PRODUCTS: Product[] = [
  {
    id: 1,
    title: 'Fjallraven - Foldsack No. 1 Backpack, Fits 15 Laptops',
    price: 109.95,
    // ...
  },
  // ... hasta el producto 20
];
```

`PRODUCTS` es una constante normal de TypeScript: **no es reactiva**. Si Angular tiene que pintar estos datos y reaccionar cuando cambien, hay que guardarlos en un signal.

2 Crear el componente `Products`

El componente se crea dentro de `src/app/features/products` con tres ficheros:

```
features/
  products/
    products.ts     # lógica (clase del componente)
    products.html   # plantilla
    products.css    # estilos
```

Se puede generar con Angular CLI:

```bash
ng generate component features/products
```

3 Crear el signal `products`

Un **signal** es un contenedor de un valor que avisa a Angular cada vez que ese valor cambia. Así Angular sabe exactamente qué parte de la pantalla tiene que volver a pintar.

En `products.ts`:

```ts
import { Component, signal } from '@angular/core';

import { PRODUCTS } from '../../core/mocks/products.mock';
import { Product } from '../../core/models/product.model';

@Component({
  selector: 'app-products',
  templateUrl: './products.html',
  styleUrl: './products.css'
})
export class Products {
  // Signal de tipo array con todos los productos del mock.
  products = signal<Product[]>(PRODUCTS);
}
```

Paso a paso:

1. **Se importa la función** `signal` desde `@angular/core`.
2. **Se importan** `PRODUCTS` **y** `Product`: el array con los datos y la interfaz que define su tipo.
3. **Se crea el signal** con `signal<Product[]>(PRODUCTS)`:
   - `signal(...)` crea el signal.
   - `<Product[]>` indica el **tipo** del valor que guarda: un array de productos. Así TypeScript solo deja guardar en él arrays de `Product`.
   - `(PRODUCTS)` es el **valor inicial**: los 20 productos del mock.

Signals y zoneless

Este proyecto funciona **sin zone.js (zoneless)**, que es el modo por defecto desde Angular 20. Sin zone.js, Angular no revisa toda la aplicación cada vez que ocurre algo (un clic, un temporizador…): solo vuelve a pintar cuando se lo notifica algo, y la forma principal de notificárselo es **cambiar un signal**.

Por eso el decorador `@Component` no necesita ninguna opción de detección de cambios (como `changeDetection: ChangeDetectionStrategy.OnPush`): con el signal es suficiente para que la plantilla se actualice cuando cambien los productos.

Leer el valor de un signal

Un signal **se lee llamándolo como una función**, con paréntesis:

```ts
this.products();   // devuelve el array Product[]
```

En la plantilla ocurre lo mismo: se escribe `products()`, no `products`.

Modificar el valor de un signal

Aunque en este paso solo se lee, conviene saber cómo se cambia:

```ts
// set: sustituye el valor por otro nuevo
this.products.set([]);

// update: calcula el nuevo valor a partir del anterior
this.products.update(list => list.filter(p => p.price < 50));
```

En ambos casos Angular se entera del cambio y vuelve a pintar la lista automáticamente.

4 La directiva `@for`

`@for` es la sintaxis de **control de flujo** de Angular para repetir un bloque de HTML por cada elemento de una colección. Sustituye a la antigua directiva `*ngFor` y **no hace falta importarla**: forma parte de la plantilla.

En `products.html`:

```HTML
<h1>Productos</h1>

<section>
  @for (product of products(); track product.id) {
    <article>
      <img [src]="product.image" [alt]="product.title" />
      <h2>{{ product.title }}</h2>
      <p>{{ product.category }}</p>
      <p>{{ product.price }} €</p>
      <p>★ {{ product.rating.rate }} ({{ product.rating.count }} valoraciones)</p>
    </article>
  } @empty {
    <p>No hay productos.</p>
  }
</section>
```

Partes de `@for`

```
@for (product of products(); track product.id) { ... }
      └─────┘    └──────────┘  └──────────────┘
     variable     colección          track
```

- `product`: variable que, en cada vuelta, contiene un producto. Solo existe dentro de las llaves `{ ... }`.
- `of products()`: la colección que se recorre. Se escriben los paréntesis porque `products` es un signal y hay que leer su valor.
- `track product.id`: **obligatorio**. Le dice a Angular cómo identificar cada elemento de forma única. Cuando la lista cambia, Angular usa este valor para saber qué cards puede reutilizar y cuáles tiene que crear o borrar, en lugar de volver a pintar todas. Lo normal es usar un campo único, como el `id`.
- `{ ... }`: el bloque de HTML que se repite. Aquí, un `<article>` (una card) por producto.

El bloque `@empty`

```HTML
} @empty {
  <p>No hay productos.</p>
}
```

`@empty` es **opcional** y va justo después del bloque de `@for`. Se muestra solo cuando la colección está vacía. Para probarlo, basta con inicializar el signal con un array vacío: `signal<Product[]>([])`.

Variables implícitas de `@for`

Dentro del bloque, Angular ofrece además unas variables ya calculadas:

Variable	Significado
`$index`	posición del elemento (empieza en 0)
`$count`	número total de elementos
`$first`	`true` si es el primer elemento
`$last`	`true` si es el último elemento
`$even`	`true` si la posición es par
`$odd`	`true` si la posición es impar

Por ejemplo, para numerar las cards:

```HTML
<h2>{{ $index + 1 }}. {{ product.title }}</h2>
```

Interpolación y property binding dentro de la card

- **Interpolación** `{{ }}`: muestra un valor como texto, por ejemplo `{{ product.title }}`.
- **Property binding** `[ ]`: asigna un valor a una propiedad de un elemento, por ejemplo `[alt]="product.title"`.
- `[src]="product.image"`: también es property binding. Asigna a la propiedad `src` de la `<img>` la ruta de la imagen de cada producto (por ejemplo `/images/1.png`, que está en la carpeta `public`). Es la etiqueta `<img>` normal de HTML, así que no hay que importar nada en el componente.

5 Estilos: el HTML sin clases

Para que el HTML se pueda leer sin ruido, **no lleva ninguna clase CSS**. Los estilos de `products.css` se aplican directamente sobre las etiquetas:

```CSS
/* Rejilla de 5 cards por fila */
section {
  display: grid;
  grid-template-columns: repeat(5, 1fr);
  gap: 1.5rem;
}

/* Card */
article {
  border: 1px solid #d1d5db;
  border-radius: 8px;
  /* ... */
}
```

Esto es posible porque Angular **encapsula** los estilos de cada componente: un selector como `section` o `article` en `products.css` solo afecta a la plantilla de `Products`, no al resto de la aplicación.

- `display: grid` convierte el `<section>` en una rejilla.
- `grid-template-columns: repeat(5, 1fr)` crea **5 columnas iguales**, así que cada fila muestra 5 cards.
- En pantallas estrechas, unas *media queries* bajan a 3, 2 y 1 columnas, para que las cards no queden demasiado pequeñas.

6 Mostrar el componente en `App`

Por último, se importa `Products` en el componente raíz y se usa su selector. Antes se borra lo que genera `ng new` por defecto: la página de bienvenida de `app.html` y el signal `title` de `app.ts`.

`app.ts`:

```ts
import { Products } from './features/products/products';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Products],
  // ...
})
export class App {}
```

`app.html`:

```html
<app-products />
<router-outlet />
```

Al arrancar la aplicación (`ng serve`) se ven los 20 productos en cards, 5 por fila.

ETIQUETA DE GIT · PASOS 7–8

02_boton_nuevo

botón **Nuevo** que muestra, con `@if`, el `div` donde se dará de alta un producto, y botón **Cancelar** que lo oculta (primer paso de la *C* de CRUD).

```
$ git checkout 02_boton_nuevo
```

7 Crear productos: planteamiento

La siguiente operación del CRUD es **crear** (*Create*) un producto nuevo. Se hará en varios pasos. El primero es preparar la pantalla:

- Un botón **Nuevo** encima del listado. Al pulsarlo aparece un `div` donde más adelante irán los campos del producto.
- Dentro del `div`, dos botones:
  - **Guardar**: de momento **no hace nada**. En un paso posterior añadirá el producto a la lista.
  - **Cancelar**: oculta el `div`.

Este paso sirve para explicar la directiva `@if`.

8 Mostrar y ocultar el div: la directiva `@if`

El signal `showForm`

Para saber si el `div` se ve o no, se añade a `products.ts` un signal de tipo `boolean`:

```ts
export class Products {
  // Signal de tipo array con todos los productos del mock.
  products = signal<Product[]>(PRODUCTS);

  // Indica si se muestra el div de alta de producto.
  showForm = signal(false);

  // Muestra el div de alta de producto.
  openForm() {
    this.showForm.set(true);
  }

  // Oculta el div de alta de producto.
  closeForm() {
    this.showForm.set(false);
  }
}
```

- No hace falta escribir `signal<boolean>(false)`: TypeScript **infiere** el tipo a partir del valor inicial `false`.
- Empieza en `false` porque, al cargar la página, el `div` está oculto.

La plantilla

En `products.html`, entre el título y el listado:

```html
<button type="button" (click)="openForm()" [attr.aria-expanded]="showForm()">Nuevo</button>

@if (showForm()) {
  <div>
    <h2>Nuevo producto</h2>
    <p>Aquí irán los campos del producto.</p>
    <button type="button">Guardar</button>
    <button type="button" (click)="closeForm()">Cancelar</button>
  </div>
}
```

`@if`

`@if` es la sintaxis de **control de flujo** de Angular para mostrar un bloque de HTML **solo si se cumple una condición**. Sustituye a la antigua directiva `*ngIf` y, igual que `@for`, **no hace falta importarla**: forma parte de la plantilla.

Partes de `@if`

```
@if (showForm()) { ... }
     └─────────┘   └─┘
      condición    bloque
```

- `(showForm())`: la **condición**, entre paréntesis. Puede ser cualquier expresión que dé `true` o `false`. Aquí se lee el signal `showForm`, por eso lleva sus propios paréntesis: `showForm()`.
- `{ ... }`: el **bloque** de HTML que se muestra cuando la condición es `true`. Aquí, el `div` con el título, el texto y los dos botones.

Otros ejemplos de condiciones válidas:

```HTML
@if (products().length > 0) { ... }
@if (product.price < 20) { ... }
@if (product.category === 'electronics') { ... }
```

`@if` crea y elimina, no oculta

- Si la condición es `true`, Angular **crea** el `div` y lo añade a la página.
- Si es `false`, el `div` **no existe** en la página: Angular lo elimina.

Es distinto de ocultar algo con CSS (`display: none`), donde el elemento sigue en el HTML pero no se ve. Con `@if`, si el `div` está cerrado, no aparece al inspeccionar la página con las herramientas del navegador. Se puede comprobar abriendo y cerrando el `div` con la pestaña *Elementos* del navegador abierta.

El bloque `@else`

`@if` admite un bloque **opcional** `@else`, que se muestra cuando la condición es `false`:

```HTML
@if (showForm()) {
  <p>El formulario está abierto.</p>
} @else {
  <p>El formulario está cerrado.</p>
}
```

El bloque `@else if`

Para comprobar varias condiciones seguidas se usa `@else if`. Angular las evalúa en orden y muestra **solo el primer bloque** cuya condición sea `true`:

```HTML
@if (product.price < 20) {
  <p>Barato</p>
} @else if (product.price < 100) {
  <p>Precio medio</p>
} @else {
  <p>Caro</p>
}
```

En este paso solo se usa `@if`, pero `@else` y `@else if` se usan igual.

Los métodos `openForm()` y `closeForm()`

Los botones no cambian el signal directamente: llaman a dos métodos de `products.ts` que lo hacen.

```ts
// Muestra el div de alta de producto.
openForm() {
  this.showForm.set(true);
}

// Oculta el div de alta de producto.
closeForm() {
  this.showForm.set(false);
}
```

- `openForm()` pone el signal a `true`, así que el `@if` muestra el `div`.
- `closeForm()` pone el signal a `false`, así que el `@if` elimina el `div`.
- Dentro de la clase, el signal se usa con `this.`: `this.showForm`.

Se usa `set` porque se sabe exactamente qué valor hay que poner en cada caso, sin depender del valor anterior.

¿Por qué llevar el código a métodos en vez de escribir `(click)="showForm.set(true)"` en la plantilla? Porque así:

- La plantilla queda más limpia y se lee mejor: `openForm()` dice **qué** hace el botón, no **cómo** lo hace.
- La lógica está en el TS. Cuando más adelante haya que hacer más cosas al cerrar el `div` (por ejemplo, vaciar los campos), solo habrá que cambiar `closeForm()`, sin tocar el HTML.

Llamar a los métodos desde los botones: event binding

```html
<button type="button" (click)="openForm()" ...>Nuevo</button>

<button type="button" (click)="closeForm()">Cancelar</button>
```

- `(click)="..."`: es un **event binding** (enlace de eventos). Los paréntesis `( )` indican que se escucha un evento del elemento, aquí el `click`, y entre comillas va lo que se ejecuta cuando ocurre: una llamada a un método del componente.
- En la plantilla los métodos se llaman **sin** `this.`: la plantilla ya tiene acceso a todo lo que hay en la clase.
- El botón **Guardar** no tiene `(click)`: de momento no hace nada.

Fíjate en la diferencia entre **leer** y **modificar** el signal:

Código	Qué hace
`showForm()`	**lee** el valor (`true` / `false`)
`showForm.set(true)`	**modifica** el valor a `true`
`showForm.set(false)`	**modifica** el valor a `false`

Los otros atributos de los botones

- `[attr.aria-expanded]="showForm()"` : property binding sobre un **atributo** HTML (de ahí el prefijo `attr.`). `aria-expanded` indica a los lectores de pantalla si el contenido que controla el botón **Nuevo** está desplegado (`true`) o no (`false`). Como lee el signal, se actualiza solo.
- `type="button"` : deja claro que son botones normales y no envían ningún formulario.

Comparando los enlaces vistos hasta ahora:

Sintaxis	Nombre	Sentido	Ejemplo
`{{ }}`	interpolación	componente → pantalla	`{{ product.title }}`
`[propiedad]`	property binding	componente → pantalla	`[src]="product.image"`
`(evento)`	event binding	pantalla → componente	`(click)="openForm()"`

Qué pasa al pulsar los botones

1. El usuario pulsa **Nuevo** y se ejecuta el método `openForm()` .
2. `openForm()` hace `this.showForm.set(true)` : el signal `showForm` pasa de `false` a `true` .
3. Como la aplicación es zoneless, el cambio del signal es lo que **avisa a Angular** de que tiene que volver a pintar.
4. Angular vuelve a evaluar las partes de la plantilla que leen `showForm()` : el `@if` crea el `div` y `aria-expanded` pasa a `true` .
5. El usuario pulsa **Cancelar**: `closeForm()` pone el signal a `false` , el `@if` elimina el `div` y `aria-expanded` vuelve a `false` .

Si se pulsa **Nuevo** con el `div` ya abierto, no cambia nada: el signal ya valía `true` .

Estilos

El HTML sigue sin clases. En `products.css` se añaden reglas para:

- `button` : todos los botones.
- `:host > div` : el `div` que es hijo directo del componente, es decir, el `div` de alta.
- `:host > div > button + button` : el botón que va justo después de otro botón dentro del `div` , es decir, **Cancelar**, para darle un estilo secundario (fondo blanco).

ETIQUETA DE GIT · PASOS 9–13

03_alta_producto

campos del producto enlazados con sus signals mediante `[value]` y `(input)`, y botón **Guardar** que añade el producto a la lista de forma inmutable (la *C* de CRUD completa).

```
$ git checkout 03_alta_producto
```

9 Los campos del producto: planteamiento

Ahora el `div` de alta tiene un campo por cada dato del producto y el botón **Guardar** ya funciona. Todavía no se usan formularios de Angular (ni `<form>` ni formularios reactivos): solo campos sueltos (`<input>` , `<select>` , `<textarea>`) enlazados con el componente.

No todos los campos de `Product` se piden al usuario:

Campo	De dónde sale
`id`	se calcula: el `id` del último producto + 1
`title`	campo de texto
`price`	campo numérico
`description`	área de texto
`category`	desplegable con los valores del enum `Category`
`image`	campo de texto con la ruta (por ejemplo `/images/3.png`)
`rating`	fijo: un producto nuevo empieza en `{ rate: 0, count: 0 }`

10 Un signal por cada campo

En `products.ts` se añade un signal por cada campo que rellena el usuario:

```ts
// Un signal por cada campo del producto nuevo.
title = signal('');
price = signal(0);
description = signal('');
category = signal(Category.Electronics);
image = signal('');
```

No hace falta escribir el tipo entre `< >` : TypeScript lo **infiere** a partir del valor inicial (`''` → `string` , `0` → `number` , `Category.Electronics` → `Category`). Para usar `Category` hay que importarlo:

```ts
import { Category } from '../../core/models/category.model';
```

¿Por qué un signal por campo y no un único signal con un objeto `Product`? Porque así cada campo se lee y se modifica de forma independiente con su propio `set`, y el código de cada campo es igual de sencillo. Con un único objeto habría que crear un objeto nuevo cada vez que el usuario escribe una letra en cualquier campo.

Para las opciones del desplegable se guarda la lista de valores del enum:

```ts
// Valores del enum Category, para las opciones del <select>.
categories = Object.values(Category);
```

`Object.values(Category)` devuelve un array con los textos del enum: `["men's clothing", "women's clothing", "jewelery", "electronics"]`. Es una propiedad normal, no un signal, porque nunca cambia.

11 Enlazar los campos con los signals: `[value]` y `(input)`

Cada campo tiene que estar enlazado con su signal **en los dos sentidos**:

- **Del componente a la pantalla**: el campo muestra el valor del signal. Se hace con **property binding**: `[value]="title()"`.
- **De la pantalla al componente**: cuando el usuario escribe, el valor se guarda en el signal. Se hace con **event binding**: `(input)="..."`, que llama a un método del TS.

Así se usan los dos enlaces que ya conocemos, sin importar nada más en el componente.

Los campos en la plantilla

El `div` sustituye el texto provisional por los campos:

```html
@if (showForm()) {
  <div>
    <h2>Nuevo producto</h2>

    <label for="title">Título</label>
    <input id="title" type="text" [value]="title()" (input)="setTitle($event)" />

    <label for="price">Precio (€)</label>
    <input id="price" type="number" min="0" step="0.01" [value]="price()"
(input)="setPrice($event)" />

    <label for="description">Descripción</label>
    <textarea id="description" rows="3" [value]="description()"
(input)="setDescription($event)"></textarea>

    <label for="category">Categoría</label>
```

```
    <select id="category" (input)="setCategory($event)">
      @for (option of categories; track option) {
        <option [value]="option" [selected]="option === category()">{{ option }}</option>
      }
    </select>

    <label for="image">Imagen</label>
    <input id="image" type="text" placeholder="/images/1.png" [value]="image()"
(input)="setImage($event)" />

    <div>
      <button type="button" (click)="addProduct()" [disabled]="!title()">Guardar</button>
      <button type="button" (click)="closeForm()">Cancelar</button>
    </div>
  </div>
}
```

Tomando el título como ejemplo:

```HTML
<input id="title" type="text" [value]="title()" (input)="setTitle($event)" />
```

- `[value]="title()"`: property binding. Asigna a la propiedad `value` del `<input>` el valor del signal `title`. Lleva paréntesis porque se **lee** el signal.
- `(input)="setTitle($event)"`: event binding. El evento `input` se produce **cada vez que cambia el contenido del campo** (cada letra que se escribe o se borra), y en ese momento se llama al método `setTitle()`.
- `$event`: variable especial de Angular que contiene el **objeto del evento** que se ha producido. Se pasa al método para que pueda saber qué campo ha cambiado y qué valor tiene.

Los métodos que modifican los signals

En `products.ts` hay un método por campo:

```ts
// Métodos que guardan en cada signal lo que escribe el usuario.
setTitle(event: Event) {
  const input = event.target as HTMLInputElement;
  this.title.set(input.value);
}

setPrice(event: Event) {
  const input = event.target as HTMLInputElement;
  this.price.set(input.valueAsNumber);
}

setDescription(event: Event) {
  const textarea = event.target as HTMLTextAreaElement;
  this.description.set(textarea.value);
}

setCategory(event: Event) {
  const select = event.target as HTMLSelectElement;
  this.category.set(select.value as Category);
}

setImage(event: Event) {
  const input = event.target as HTMLInputElement;
  this.image.set(input.value);
}
```

Paso a paso, con `setTitle`:

1. `event: Event`: el parámetro recibe el `$event` de la plantilla. Su tipo es `Event`, el tipo de TypeScript para cualquier evento del navegador.
2. `event.target`: es el **elemento HTML que ha producido el evento**, en este caso el `<input>` del título.
3. `as HTMLInputElement`: TypeScript solo sabe que `event.target` es "algo que produce eventos" (`EventTarget`), que no tiene la propiedad `value`. Con `as` se le **indica el tipo concreto**: es un `<input>`, es decir, un `HTMLInputElement`. Esto se llama **aserción de tipo** (*type assertion*). Cada elemento HTML tiene su tipo:

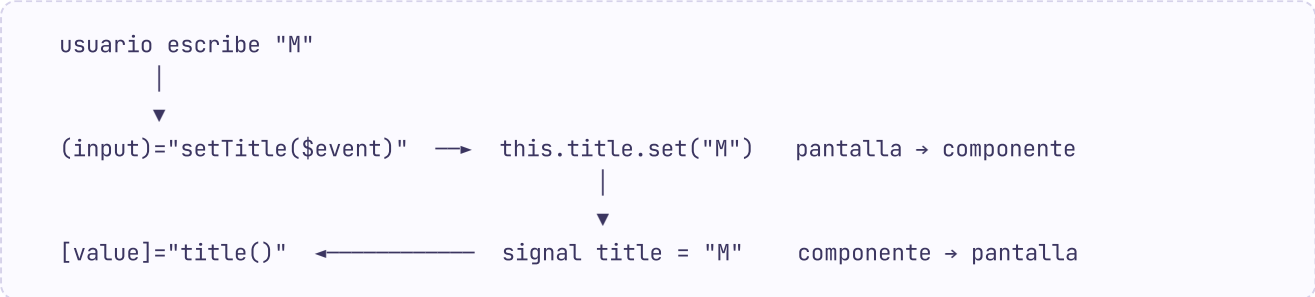
Elemento	Tipo de TypeScript
`<input>`	`HTMLInputElement`
`<textarea>`	`HTMLTextAreaElement`
`<select>`	`HTMLSelectElement`

4. `input.value`: el texto que hay escrito en el campo.
5. `this.title.set(...)`: se guarda ese texto en el signal.

Los demás métodos siguen la misma estructura, con estas diferencias:

- `setPrice` usa `input.valueAsNumber` en vez de `input.value`. `value` siempre es un texto (`"12.5"`), y el signal `price` es un `number`. `valueAsNumber` devuelve el contenido de un campo `type="number"` ya convertido a número (`12.5`). Si el campo está vacío devuelve `NaN` (*Not a Number*), un caso que se trata al guardar (paso 13).
- `setCategory` usa `select.value as Category`. El valor de un `<select>` es un `string`, pero el signal es de tipo `Category`. Como las opciones salen de los valores del enum, sabemos que el texto es una categoría válida, y se le indica a TypeScript con `as Category`.

El ciclo completo



Los dos sentidos son necesarios:

- Sin `(input)`, el usuario escribiría pero el signal no se enteraría y, al guardar, el producto tendría el título vacío.
- Sin `[value]`, el campo no mostraría el valor del signal. Se nota al cerrar el `div` y volver a abrirlo: los campos tienen que aparecer con el valor que tengan los signals en ese momento (vacíos, porque `closeForm()` los reinicia).

El desplegable de categorías

```HTML
<select id="category" (input)="setCategory($event)">
  @for (option of categories; track option) {
    <option [value]="option" [selected]="option === category()">{{ option }}</option>
  }
</select>
```

- Se vuelve a usar `@for`, esta vez para crear un `<option>` por cada categoría. Como cada categoría es un texto distinto, se usa el propio valor en el `track`.
- La variable del bucle se llama `option` y no `category` para no confundirla con el signal `category` del componente.
- `[value]="option"`: el valor de cada opción es el texto de la categoría.
- `[selected]="option === category()"`: en un `<select>` el sentido "componente → pantalla" se hace marcando como **seleccionada** la opción cuyo valor coincide con el del signal. Por eso al abrir el `div` aparece elegida `electronics`, el valor inicial del signal.
- `(input)="setCategory($event)"`: al elegir otra opción se produce el evento `input` y se guarda la categoría en el signal.

Otros detalles de los campos

- `<label for="...">`: cada campo lleva su etiqueta con el atributo `for` igual al `id` del campo. Así los lectores de pantalla leen la etiqueta al llegar al campo y, al pulsar sobre el texto de la etiqueta, el foco va al campo.
- `[disabled]="!title()"`: property binding que desactiva **Guardar** mientras el título esté vacío, para no crear productos sin nombre. Como `title` es un signal, el botón se activa en cuanto el usuario escribe la primera letra.
- Los botones van dentro de otro `div` para colocarlos uno al lado del otro con CSS (`:host > div > div`).

Tabla de los enlaces usados hasta ahora:

Sintaxis	Nombre	Sentido	Ejemplo
`{{ }}`	interpolación	componente → pantalla	`{{ product.title }}`
`[propiedad]`	property binding	componente → pantalla	`[value]="title()"`
`(evento)`	event binding	pantalla → componente	`(input)="setTitle($event)"`

12 Modificar un signal de tipo array: la inmutabilidad

Este es el punto más importante de esta parte.

Cómo sabe un signal que ha cambiado

Cuando se llama a `set` o a `update`, el signal compara el valor nuevo con el anterior. Si **son el mismo**, considera que no ha cambiado nada y **no avisa a Angular**, así que la pantalla no se vuelve a pintar.

Con números, textos o booleanos, la comparación es por valor: `3` y `4` son distintos, `false` y `true` también. Pero con arrays y objetos se compara la **referencia**, es decir, si son **el mismo array en memoria**, no si tienen los mismos elementos.

El error: modificar el array con `push`

```ts
// ❌ MAL
this.products.update(products => {
  products.push(newProduct);
  return products;
});
```

`push` **modifica (muta) el array que ya existe**: le añade un elemento, pero sigue siendo el mismo array. Al devolverlo, el signal compara el array "nuevo" con el anterior, ve que es **el mismo** y no avisa a nadie. El producto está dentro del array, pero **la pantalla no cambia**.

Lo mismo pasa con cualquier método que modifica el array original: `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, o asignar `products[0] = ...`.

La solución: inmutabilidad

Inmutabilidad significa **no modificar nunca un dato que ya existe, sino crear uno nuevo con los cambios**. En vez de añadir el producto al array actual, se crea un array nuevo que contiene los productos anteriores más el nuevo:

```ts
// ✅ BIEN
this.products.update(products => [...products, newProduct]);
```

- `update` recibe una función: su parámetro (`products`) es el valor actual del signal, y lo que devuelve es el nuevo valor.
- `[ ... ]` crea un **array nuevo**.
- `...products` es el **operador spread** (propagación): copia dentro del array nuevo todos los elementos del array anterior.
- `, newProduct` añade el producto nuevo al final.

Gráficamente:

```
antes:   products ──► [ p1, p2, ..., p20 ]

después: products ──► [ p1, p2, ..., p20, p21 ]   ← array NUEVO
                      [ p1, p2, ..., p20 ]        ← el anterior no se toca
```

Ahora el signal compara dos arrays **distintos**, detecta el cambio y avisa a Angular, que pinta la nueva card. Gracias al `track product.id` del `@for`, Angular reutiliza las 20 cards que ya tenía y solo crea la nueva.

El array anterior no se modifica, solo se deja de usar. Esto hace el código más predecible: un dato que ya existe nunca cambia "por sorpresa".

`set` o `update`

Las dos formas son correctas siempre que se cree un array nuevo:

```ts
// update: el nuevo valor se calcula a partir del anterior
this.products.update(products => [...products, newProduct]);

// set: equivalente, leyendo antes el valor actual
this.products.set([...this.products(), newProduct]);
```

Se prefiere `update` cuando el nuevo valor depende del anterior, como aquí. `set` se usa cuando el valor nuevo no depende del anterior, como al vaciar un campo: `this.title.set('')`.

Adelanto: el resto del CRUD

Las siguientes operaciones seguirán la misma regla: **crear siempre un array nuevo**. Para cada una hay una forma que no modifica el array original:

Operación	Forma	Devuelve
Añadir	`[...array, x]`	un array nuevo con un elemento más
Borrar	`array.filter()`	un array nuevo sin los elementos filtrados
Editar	`array.map()`	un array nuevo con algún elemento cambiado

13 Guardar el producto y cerrar el div

El método `addProduct()`

```ts
// Crea el producto nuevo y lo añade a la lista.
addProduct() {
  // El id es el del último producto + 1 (o 1 si la lista está vacía).
  const lastProduct = this.products().at(-1);
  const id = lastProduct ? lastProduct.id + 1 : 1;

  const newProduct: Product = {
    id: id,
    title: this.title(),
    // Si el campo del precio está vacío, su valor es NaN: se guarda 0.
    price: this.price() || 0,
    description: this.description(),
    category: this.category(),
    image: this.image(),
    rating: { rate: 0, count: 0 }
  };

  // Inmutabilidad: no se modifica el array actual, se crea uno nuevo.
  this.products.update(products => [...products, newProduct]);

  this.closeForm();
}
```

1. **Se calcula el** `id` a partir del último producto de la lista:
   - `this.products().at(-1)` devuelve el **último elemento** del array. Los índices negativos de `at()` cuentan desde el final: `at(-1)` es el último, `at(-2)` el penúltimo... Más abajo se explica con detalle y se compara con otras formas de obtener el último elemento.
   - Si la lista está vacía, `at(-1)` devuelve `undefined`. Por eso se comprueba con el operador ternario `condición ? valorSiTrue : valorSiFalse`: si hay último producto, su `id` + 1; si no, `1`.
   - Con los 20 productos del mock, el último tiene `id` 20, así que el nuevo tendrá el `id` 21.

Esta forma de calcular el `id` funciona porque la lista **siempre está ordenada por** `id`: el mock viene ordenado y los productos nuevos se añaden siempre **al final**. Así, el último producto es siempre el de mayor `id` y nunca se repite ninguno.

2. **Se crea el objeto** `newProduct` de tipo `Product`. Si faltara algún campo de la interfaz, TypeScript daría un error. Los valores se leen de cada signal **llamándolo**: `this.title()`, `this.price()`...

   En el precio se escribe `this.price() || 0`. Si el usuario deja el campo vacío, `valueAsNumber` devuelve `NaN` (paso 11). El operador `||` devuelve el valor de la izquierda si es "verdadero" y, si no, el de la derecha. `NaN` cuenta como falso, así que en ese caso se guarda `0`.

3. **Se añade al signal** `products` **de forma inmutable**, con `update` y el operador spread (paso 12).

4. **Se cierra el** `div` llamando a `closeForm()`.

Obtener el último producto: `at(-1)` y alternativas

`at()` es un método de los arrays que devuelve el elemento de una posición. A diferencia de los corchetes, admite **índices negativos**, que cuentan desde el final:

```ts
const letters = ['a', 'b', 'c', 'd'];

letters.at(0);   // 'a'  (igual que letters[0])
letters.at(-1);  // 'd'  el último
letters.at(-2);  // 'c'  el penúltimo
letters.at(10);  // undefined: no existe esa posición

[].at(-1);       // undefined: el array está vacío
```

Hay otras formas de obtener el último elemento. Se comparan aquí para entender por qué se ha elegido `at(-1)`:

```ts
const products = this.products();

// ✅ 1. at(-1): la que se usa en el proyecto
const lastProduct = products.at(-1);

// ✅ 2. Índice length - 1: la forma clásica
const lastProduct = products[products.length - 1];

// ⚠️ 3. slice(-1) + desestructuración
const [lastProduct] = products.slice(-1);

// ⚠️ 4. findLast(): el último que cumple una condición (ES2023)
const lastProduct = products.findLast(() => true);

// ❌ 5. pop(): devuelve el último, pero lo QUITA del array
const lastProduct = products.pop();
```

1. `at(-1)`: es la más corta y la que mejor se lee ("el elemento en la posición -1, la última"). Además, TypeScript sabe que el resultado es de tipo `Product | undefined`: si la lista estuviera vacía

devolvería `undefined`, y TypeScript **obliga a comprobarlo** antes de usar `lastProduct.id`. Por eso `addProduct()` hace `lastProduct ? lastProduct.id + 1 : 1`. Existe desde ES2022, la versión de JavaScript para la que compila el proyecto.

2. `products[products.length - 1]`: es la forma que aparece en la mayoría del código antiguo. Como los índices empiezan en 0, el último es `length - 1` (con 20 productos, el índice 19). Funciona igual, pero:
   - hay que escribir el nombre del array dos veces; con el signal directamente quedaría `this.products()[this.products().length - 1]`;
   - TypeScript considera que el resultado es `Product`, aunque con la lista vacía sea `undefined`, así que **no avisa** si se olvida esa comprobación.
3. `slice(-1)`: `slice` devuelve un **array nuevo** con el último elemento (sin modificar el original) y `[lastProduct]` saca su primer elemento (*desestructuración*). Funciona, pero usa dos conceptos para algo tan sencillo como leer un elemento y, como con los corchetes, TypeScript no avisa de que puede ser `undefined`.
4. `findLast()`: recorre el array desde el final y devuelve el primer elemento que cumple la condición. Con `() => true` devuelve el último, pero es dar un rodeo. Tiene sentido cuando sí hay una condición, por ejemplo el último producto de joyería: `products.findLast(product => product.category === Category.Jewelery)`. Ojo: es de **ES2023** y este proyecto compila para ES2022, así que TypeScript da un error si se usa sin cambiar la opción `lib` del `tsconfig.json`.
5. `pop()`: **no se debe usar aquí**. Devuelve el último elemento, pero **lo elimina del array**: modifica (muta) el array del signal, justo lo contrario de la inmutabilidad del paso 12. El último producto desaparecería de la lista y, además, el signal no avisaría a Angular.

Forma	¿Modifica el array?	Si la lista está vacía	¿TypeScript avisa del `undefined`?
`at(-1)`	No	`undefined`	Sí
`[length - 1]`	No	`undefined`	No
`slice(-1)`	No	`undefined`	No
`findLast(() => true)`	No	`undefined`	Sí (requiere ES2023)
`pop()`	**Sí** ❌	`undefined`	Sí

`closeForm()` ahora también vacía los campos

```ts
// Vacía los campos y oculta el div de alta de producto.
closeForm() {
  this.title.set('');
  this.price.set(0);
  this.description.set('');
  this.category.set(Category.Electronics);
  this.image.set('');
  this.showForm.set(false);
}
```

Antes `closeForm()` solo ponía `showForm` a `false`. Ahora, además, devuelve cada signal a su valor inicial con `set`, para que al pulsar **Nuevo** otra vez los campos aparezcan vacíos. Como lo usan tanto **Guardar** (desde `addProduct()`) como **Cancelar**, el cambio solo se ha hecho en un sitio: es la ventaja de haber puesto la lógica en un método.

Al volver a abrir el `div`, el `@if` crea los campos de nuevo y cada `[value]` muestra el valor de su signal: vacío, porque `closeForm()` los ha reiniciado. Es el sentido "componente → pantalla" del enlace.

Qué pasa al pulsar Guardar

1. El usuario pulsa **Nuevo**: `openForm()` muestra el `div`.
2. Escribe en los campos: en cada cambio, el evento `(input)` llama a su método (`setTitle()`, `setPrice()`...), que guarda lo escrito en su signal con `set`.
3. Pulsa **Guardar** y se ejecuta `addProduct()`:
   - calcula el `id` (21);
   - crea `newProduct` leyendo los signals de los campos;
   - crea un array nuevo con los 20 productos más el nuevo y lo guarda en `products` con `update`.
4. El signal `products` ha cambiado (es otro array), así que avisa a Angular y el `@for` pinta la card número 21 al final de la lista.
5. `closeForm()` vacía los signals de los campos y pone `showForm` a `false`: el `@if` elimina el `div`.

Los productos nuevos solo existen mientras la página está abierta: al recargar, la lista vuelve a los 20 productos del mock.

ETIQUETA DE GIT · PASOS 14–15

04_borrado_producto

botón **Borrar** en cada card que elimina el producto de forma inmutable con `filter` (la *D* de CRUD).

```
$ git checkout 04_borrado_producto
```

14 Borrar productos: planteamiento

La siguiente operación del CRUD es **borrar** (*Delete*). Cada card tiene un botón **Borrar** que, al pulsarlo, llama a un método del componente que elimina ese producto de la lista.

Para borrar se sigue la misma regla que para añadir (paso 12): **no se modifica el array que ya existe, se crea uno nuevo** sin el producto borrado.

15 El botón Borrar y el método `deleteProduct()`

El botón en cada card

En `products.html`, dentro del `<article>` de cada producto, después de la valoración:

HTML

```
@for (product of products(); track product.id) {
  <article>
    <img [src]="product.image" [alt]="product.title" />
    <h2>{{ product.title }}</h2>
    <p>{{ product.category }}</p>
    <p>{{ product.price }} €</p>
    <p>★ {{ product.rating.rate }} ({{ product.rating.count }} valoraciones)</p>
    <button type="button" (click)="deleteProduct(product.id)" [attr.aria-label]="'Borrar ' +
product.title">Borrar</button>
  </article>
}
```

- Como el botón está **dentro del** `@for`, se crea un botón por cada card.
- `(click)="deleteProduct(product.id)"`: event binding que llama al método **pasándole un argumento**: el `id` del producto de esa card. Cada botón tiene acceso a la variable `product` de su vuelta del `@for`, así que el botón de la primera card envía `1`, el de la segunda `2`, etc.
- Se pasa el `id` y no el producto entero porque es lo único que hace falta para saber qué producto borrar: el `id` es único.

- `[attr.aria-label]="'Borrar ' + product.title"`: todos los botones dicen "Borrar", y un lector de pantalla que los recorra oiría "Borrar, Borrar, Borrar..." sin saber de qué producto es cada uno. `aria-label` da al botón un nombre accesible más completo, por ejemplo *"Borrar Mens Cotton Jacket"*. En pantalla se sigue viendo "Borrar". Es un property binding sobre un atributo (`attr.`), y su valor es una expresión que une el texto `'Borrar '` con el título del producto.

El método `deleteProduct()`

En `products.ts`:

```ts
// Borra el producto con el id indicado.
deleteProduct(id: number) {
  // Inmutabilidad: filter crea un array nuevo sin el producto borrado.
  this.products.update(products => products.filter(product => product.id !== id));
}
```

- `id: number`: el parámetro recibe el `id` que envía el botón.
- `this.products.update(...)`: igual que al añadir, el nuevo valor del signal se calcula a partir del anterior.
- `products.filter(...)`: crea un **array nuevo** con los elementos que cumplen la condición.
- `product => product.id !== id`: la condición. Se quedan todos los productos cuyo `id` **sea distinto** del que se quiere borrar. Es decir, se quedan todos menos ese.

Cómo funciona `filter`

`filter` recorre el array y, para cada elemento, ejecuta la función que se le pasa:

- Si la función devuelve `true`, el elemento **se queda** en el array nuevo.
- Si devuelve `false`, el elemento **no se copia** al array nuevo.

Por ejemplo, al borrar el producto con `id` 3:

```
products:  [ p1, p2, p3, p4, p5, ... ]
             |   |   |   |   |
id !== 3:  true true false true true ...
             |   |       |   |
nuevo:     [ p1, p2,     p4, p5, ... ]   ← array NUEVO, sin p3
```

Lo importante es que `filter` **no modifica el array original**: devuelve uno nuevo. El signal compara el array nuevo con el anterior, ve que son distintos y avisa a Angular, que quita la card de la pantalla. Gracias al `track product.id`, Angular sabe exactamente qué card ha desaparecido y **solo elimina esa**, sin volver a pintar las demás.

El error: borrar con `splice`

Los arrays tienen un método para quitar elementos, `splice`, pero **modifica el array original**:

```ts
// ❌ MAL
deleteProduct(id: number) {
  this.products.update(products => {
    const index = products.findIndex(product => product.id === id);
    products.splice(index, 1);
    return products;
  });
}
```

Es el mismo problema que con `push` (paso 12): el array devuelto es **el mismo** que había, el signal no detecta el cambio y **la card no desaparece de la pantalla**, aunque el producto ya no esté en el array. Además, el código es más largo: hay que buscar primero la posición del producto.

Comparación de las operaciones vistas:

Operación	❌ Muta el array	✅ Crea un array nuevo
Añadir	`products.push(p)`	`[...products, p]`
Borrar	`products.splice(i, 1)`	`products.filter(...)`

Cuando se borran todos los productos

Si se borran todos, el signal `products` pasa a ser un array vacío `[]` y el `@for` muestra su bloque `@empty` (paso 4):

```html
} @empty {
  <p>No hay productos.</p>
}
```

Si después se da de alta un producto, `addProduct()` no encuentra último producto (`at(-1)` devuelve `undefined`) y le asigna el `id` 1: es el caso de la lista vacía que se tuvo en cuenta en el paso 13.

El `id` de los nuevos productos después de borrar

Como el `id` de un producto nuevo es el del **último** + 1:

- Si se borra un producto **del medio** (por ejemplo el 5), el último sigue siendo el 20 y el nuevo será el 21. No se repite ningún `id` ; simplemente queda un "hueco" en el 5, y eso no es un problema: un `id` tiene que ser único, no consecutivo.
- Si se borra **el último** (el 20), el nuevo último es el 19 y el siguiente producto que se cree tendrá el `id` 20. Ese `id` se **reutiliza**, pero no se repite, porque el producto que lo tenía ya no existe.

Qué pasa al pulsar Borrar

1. El usuario pulsa **Borrar** en la card del producto con `id` 3.
2. Se ejecuta `deleteProduct(3)` .

3. `filter` crea un array nuevo con todos los productos menos el 3, y `update` lo guarda en el signal `products`.
4. El signal ha cambiado (es otro array), así que avisa a Angular.
5. El `@for` compara la lista nueva con la anterior usando el `track product.id` y elimina solo la card del producto 3.

Estilos

El botón **Borrar** usa el selector `article button` (los botones que están dentro de una card) para darle color rojo, que indica una acción destructiva. Con `margin-top: auto` se coloca al final de la card, así que todos los botones quedan alineados abajo aunque los títulos tengan distinta longitud.

Los productos borrados, igual que los añadidos, solo cambian mientras la página está abierta: al recargar, la lista vuelve a los 20 productos del mock.

ETIQUETA DE GIT · PASOS 16–17

05_boton_editar

botón **Editar** en cada card que abre el `div` en modo edición con los datos del producto (primer paso de la *U* de CRUD).

```
$ git checkout 05_boton_editar
```

16 Editar productos: planteamiento

La última operación del CRUD es **editar** (*Update*). En lugar de crear otro `div` con los mismos campos, se **reutiliza el** `div` **del alta**, que ahora puede estar en dos modos:

- **Nuevo**: se abre con el botón **Nuevo**, con los campos vacíos y el título "Nuevo producto".
- **Edición**: se abre con un botón **Editar** que hay en cada card, con los campos rellenos con los datos de ese producto y el título "Editar producto".

La edición se hace en dos pasos:

- **Este paso**: el botón **Editar** abre el `div` en modo edición con los datos del producto cargados.
- **Paso siguiente**: el botón **Guardar** sabrá si tiene que crear o actualizar.

En este paso **Guardar sigue llamando a** `addProduct()` : si se pulsa en modo edición, **crea un producto nuevo** con los datos del formulario en vez de modificar el existente. Se corrige en el paso siguiente.

17 El modo del div y el producto seleccionado

Dos signals nuevos

En `products.ts` :

```ts
// Modo del div: crear un producto nuevo o editar uno existente.
mode = signal<'new' | 'edit'>('new');

// Producto que se está editando (undefined si se está creando uno nuevo).
selectedProduct = signal<Product | undefined>(undefined);
```

- `mode` indica **qué** se está haciendo en el `div` : crear (`'new'`) o editar (`'edit'`).
- `selectedProduct` indica **sobre qué producto**: el que se está editando, o `undefined` si se está creando uno nuevo.

Tipos unión

En los dos signals **hay que escribir el tipo** entre `< >`. Si no, TypeScript lo deduciría del valor inicial y sería demasiado limitado.

`'new' | 'edit'` es un **tipo unión de literales**:

- El `|` se lee "o": el valor puede ser `'new'` **o** `'edit'`.
- Son **literales**: no vale cualquier `string`, solo esos dos textos exactos.

```ts
this.mode.set('edit');    // ✅
this.mode.set('editar');  // ❌ error: '"editar"' no es '"new" | "edit"'
```

Sin el tipo, `signal('new')` sería de tipo `string` y aceptaría cualquier texto, incluido uno mal escrito.

Se ha elegido `'edit'` y no `'update'` para no confundirlo con el método `update()` de los signals, que se usa en este mismo componente (`this.products.update(...)`).

`Product | undefined` es una unión de un tipo con `undefined`: el signal contiene un `Product` **o** nada. Es el mismo tipo que devuelve `at(-1)` (paso 13) y, como allí, TypeScript **obligará a comprobar** que no es `undefined` antes de usar sus propiedades. Sin el tipo, `signal(undefined)` solo podría contener `undefined`, y nunca se le podría asignar un producto.

El botón Editar en cada card

```html
<div>
  <button type="button" (click)="editProduct(product)" [attr.aria-label]="'Editar ' + product.title">Editar</button>
  <button type="button" (click)="deleteProduct(product.id)" [attr.aria-label]="'Borrar ' + product.title">Borrar</button>
</div>
```

- Los dos botones van dentro de un `div` para colocarlos uno al lado del otro al final de la card.
- `(click)="editProduct(product)"`: a diferencia de Borrar, que solo envía el `id`, Editar envía **el producto entero**, porque hacen falta todos sus datos para rellenar los campos.
- `[attr.aria-label]`: igual que en Borrar, para que un lector de pantalla diga *"Editar Mens Cotton Jacket"* y no solo "Editar".

El método `editProduct()`

```ts
// Muestra el div para editar el producto seleccionado, con sus datos cargados.
editProduct(product: Product) {
  this.mode.set('edit');
  this.selectedProduct.set(product);

  this.title.set(product.title);
  this.price.set(product.price);
  this.description.set(product.description);
  this.category.set(product.category);
  this.image.set(product.image);

  this.showForm.set(true);
}
```

1. Pone el `div` en modo edición.
2. Guarda el producto en `selectedProduct`, para saber más adelante cuál hay que actualizar.
3. Copia cada dato del producto en el signal de su campo. Como los campos leen los signals con `[value]` (paso 11), **aparecen rellenos**. En el `<select>`, `[selected]` marca la categoría del producto.
4. Abre el `div`.

Si el `div` ya estaba abierto (editando otro producto o creando uno nuevo), `@if` no lo vuelve a crear, pero los `[value]` se actualizan con los nuevos valores de los signals, así que los campos cambian a los datos del nuevo producto.

Renombrar `openForm()` a `newProduct()`

Hasta el paso anterior, el botón **Nuevo** llamaba a `openForm()`. El nombre tenía sentido porque solo había una forma de abrir el `div`: para crear un producto.

Ahora el `div` se abre de **dos formas**, cada una desde un botón y con un método distinto:

Botón	Método	Cómo deja el `div`
Nuevo	`newProduct()`	campos vacíos, modo `'new'`, sin producto seleccionado
Editar (en cada card)	`editProduct(product)`	campos rellenos, modo `'edit'`, producto seleccionado

Con el nombre `openForm()`, al leer el código parecería que ese método sirve para abrir el `div` **en cualquier caso**, también para editar, y no es así: solo lo prepara para un producto nuevo. Por eso se cambia a `newProduct()`:

- Dice **para qué** sirve el botón (crear un producto nuevo), no solo qué hace con el `div` (abrirlo).
- Forma pareja con `editProduct()`: un método por cada forma de abrir el `div`.

Un buen nombre de método describe **la acción del usuario o la intención**, no solo el detalle técnico. Cuando un método empieza a hacer más cosas o aparecen otros parecidos, conviene revisar si su nombre sigue siendo claro.

El cambio afecta a dos ficheros.

En `products.ts`, el nombre del método:

```ts
// Antes (pasos 8 a 15)
openForm() {
  this.showForm.set(true);
}

// Ahora
newProduct() {
  this.clearFields();
  this.mode.set('new');
  this.selectedProduct.set(undefined);
  this.showForm.set(true);
}
```

En `products.html`, la llamada del botón **Nuevo**:

```html
<!-- Antes -->
<button type="button" (click)="openForm()" [attr.aria-expanded]="showForm()">Nuevo</button>

<!-- Ahora -->
<button type="button" (click)="newProduct()" [attr.aria-
expanded]="showForm()">Nuevo</button>
```

Si se cambia el nombre en el TS y se olvida en el HTML, Angular da un error al compilar, porque la plantilla llama a un método que ya no existe:

```
Property 'openForm' does not exist on type 'Products'.
```

Para renombrar sin olvidar ningún sitio, en VS Code se puede poner el cursor sobre el nombre del método y pulsar **F2** (*Rename Symbol*): cambia el nombre en el TS y, con la extensión *Angular Language Service*, también en la plantilla.

En los pasos 8 a 15 de esta guía el método sigue apareciendo como `openForm()`, porque así se llamaba en esos commits.

Renombrar la constante `newProduct` a `product` en `addProduct()`

Al llamar `newProduct()` al método, aparece un problema en `addProduct()` (paso 13): dentro de ese método había una **constante local** que se llamaba igual:

```ts
// Antes (pasos 13 a 15)
addProduct() {
  // ...
  const newProduct: Product = {
    id: id,
    title: this.title(),
    // ...
  };

  this.products.update(products => [...products, newProduct]);

  this.closeForm();
}
```

El código **seguiría funcionando**, porque son dos cosas distintas que viven en sitios distintos:

- `this.newProduct` es el **método** del componente. Para usarlo desde la clase siempre hay que escribir `this.` delante: `this.newProduct()`.
- `newProduct`, sin `this.`, es la **constante local** de `addProduct()`. Solo existe dentro de ese método, desde que se declara hasta la llave de cierre `}`. A eso se le llama su **ámbito** (*scope*).

Pero tener dos cosas con el mismo nombre **confunde al leer el código**: al ver `newProduct` dentro de `addProduct()` hay que pararse a pensar si se refiere al método o a la constante. Por eso la constante se renombra a `product`:

```ts
// Ahora
addProduct() {
  // El id es el del último producto + 1 (o 1 si la lista está vacía).
  const lastProduct = this.products().at(-1);
  const id = lastProduct ? lastProduct.id + 1 : 1;

  const product: Product = {
    id: id,
    title: this.title(),
    // Si el campo del precio está vacío, su valor es NaN: se guarda 0.
    price: this.price() || 0,
    description: this.description(),
    category: this.category(),
    image: this.image(),
    rating: { rate: 0, count: 0 }
  };

  // Inmutabilidad: no se modifica el array actual, se crea uno nuevo.
  this.products.update(products => [...products, product]);

  this.closeForm();
}
```

Hay que cambiarla **en los dos sitios** donde aparece: donde se declara (`const product: Product = {...}`) y donde se usa (`[...products, product]`). Con **F2** en VS Code también se renombran las dos a la vez.

No hay confusión con otros `product` del componente:

- En `deleteProduct()`, `product` es el parámetro de la función flecha de `filter` (`product => product.id !== id`) y solo existe dentro de esa función.
- En la plantilla, `product` es la variable del `@for`, que solo existe dentro de su bloque.

Cada uno vive en su propio ámbito y no se mezclan. Lo que se ha evitado es que **un mismo nombre** se refiera a un método del componente y a una variable a la vez.

En los pasos 13 a 15 de esta guía la constante sigue apareciendo como `newProduct`, porque así se llamaba en esos commits.

`clearFields()`, `newProduct()` y `closeForm()`

Al añadir el modo edición, el botón **Nuevo** también tiene que preparar el `div`: si antes se estaba editando, hay que volver al modo nuevo y vaciar los campos. Como `closeForm()` ya vaciaba los campos, ese código se lleva a un método propio para no repetirlo:

```ts
// Vacía los signals de los campos.
clearFields() {
  this.title.set('');
  this.price.set(0);
  this.description.set('');
  this.category.set(Category.Electronics);
  this.image.set('');
}

// Muestra el div para crear un producto nuevo, con los campos vacíos.
newProduct() {
  this.clearFields();
  this.mode.set('new');
  this.selectedProduct.set(undefined);
  this.showForm.set(true);
}

// Vacía los campos, vuelve al modo nuevo y oculta el div.
closeForm() {
  this.clearFields();
  this.mode.set('new');
  this.selectedProduct.set(undefined);
  this.showForm.set(false);
}
```

Cambiar siempre los dos signals juntos

`mode` y `selectedProduct` están relacionados:

`mode`	`selectedProduct`
`'new'`	`undefined`
`'edit'`	un `Product`

Por eso **todos los métodos que cambian uno cambian también el otro**: `newProduct()`, `editProduct()` y `closeForm()`. Si alguno se olvidara, podría quedar, por ejemplo, `mode` en `'edit'` sin producto seleccionado, y el `div` diría "Editar producto" sin saber cuál.

Más adelante se verá `computed()`, que permite **calcular** un signal a partir de otro (por ejemplo, el modo a partir de si hay producto seleccionado) para que nunca puedan desincronizarse.

El título del div: `@if` con `@else`

```HTML
@if (mode() === 'new') {
  <h2>Nuevo producto</h2>
} @else {
  <h2>Editar producto</h2>
}
```

Aquí se usa por primera vez el bloque `@else` explicado en el paso 8:

- Si `mode()` vale `'new'`, se muestra "Nuevo producto".
- En cualquier otro caso (es decir, `'edit'`), se muestra "Editar producto".

`===` compara si dos valores son **exactamente iguales**. Como `mode` es de tipo `'new' | 'edit'`, TypeScript también comprueba esta comparación: si se escribiera `mode() === 'nuevo'`, daría error porque nunca podría ser cierta.

Qué pasa al pulsar Editar

1. El usuario pulsa **Editar** en la card de "Mens Cotton Jacket" (`id` 3).
2. Se ejecuta `editProduct(product)` con ese producto.
3. `mode` pasa a `'edit'` y `selectedProduct` guarda el producto 3.
4. Los signals de los campos reciben sus datos.
5. `showForm` pasa a `true` y el `@if` crea el `div`.
6. El título muestra "Editar producto" y los campos aparecen rellenos.
7. **Cancelar** llama a `closeForm()`: vacía los campos, vuelve al modo nuevo, quita el producto seleccionado y cierra el `div`.

Como se ha dicho en el paso 16, en este paso **Guardar** todavía crea un producto nuevo también en modo edición.

Estilos

Los botones de la card están dentro de un `div`, así que los selectores cambian:

- `article > div`: el `div` de los botones, al final de la card (`margin-top: auto`) y con los botones uno al lado del otro.
- `article > div > button + button`: el segundo botón, **Borrar**, en rojo. **Editar** usa el estilo azul de todos los botones.

ETIQUETA DE GIT · PASOS 18–19

06_guardar_edicion

Guardar crea o actualiza según el modo; la edición sustituye el producto con `map` y el spread de objetos (la *U* de CRUD completa).

```
$ git checkout 06_guardar_edicion
```

18 Guardar la edición: planteamiento

En el paso anterior, **Guardar** siempre llamaba a `addProduct()`. En modo edición eso creaba **una copia** del producto al final de la lista (con `id` nuevo y la valoración a 0) y el original no cambiaba.

En este paso:

- **Guardar** llama a un método `save()` que, según el modo, **crea** un producto nuevo o **actualiza** el seleccionado.
- El nuevo método `updateProduct()` sustituye el producto en la lista, de forma inmutable, con `map()` y el **spread de objetos**.
- Si se borra el producto que se está editando, se cierra el `div`.

Con esto el CRUD queda completo: **C**reate (crear), **R**ead (leer), **U**pdate (editar) y **D**elete (borrar).

19 `save()`, `updateProduct()` y la inmutabilidad en objetos

Guardar llama a `save()`

En `products.html`, el botón **Guardar** ya no llama directamente a `addProduct()`:

HTML

```
<!-- Antes -->
<button type="button" (click)="addProduct()" [disabled]="!title()">Guardar</button>

<!-- Ahora -->
<button type="button" (click)="save()" [disabled]="!title()">Guardar</button>
```

Y en `products.ts`:

```ts
// Guardar: según el modo, crea un producto nuevo o actualiza el seleccionado.
save() {
  if (this.mode() === 'new') {
    this.addProduct();
  } else {
    this.updateProduct();
  }
}
```

- Se lee el signal `mode` (con paréntesis) y se compara con `'new'`.
- Si es `'new'`, se crea un producto con `addProduct()`, que no cambia.
- Si no (es decir, `'edit'`), se actualiza con `updateProduct()`.

La decisión se toma **en el TS** y no en la plantilla. Se podría escribir `(click)="mode() === 'new' ? addProduct() : updateProduct()"`, pero la plantilla quedaría más difícil de leer: es la misma idea que llevó a crear `openForm()` y `closeForm()` en el paso 8.

El método `updateProduct()`

```ts
// Actualiza el producto seleccionado con los datos de los campos.
updateProduct() {
  const selected = this.selectedProduct();
  if (!selected) {
    return;
  }

  // Inmutabilidad: se crea un objeto nuevo copiando el producto seleccionado
  // (conserva id y rating) y sustituyendo los datos que se pueden editar.
  const updatedProduct: Product = {
    ...selected,
    title: this.title(),
    price: this.price() || 0,
    description: this.description(),
    category: this.category(),
    image: this.image()
  };

  // Inmutabilidad: map crea un array nuevo con el producto actualizado en su sitio.
  this.products.update(products =>
    products.map(product => (product.id === updatedProduct.id ? updatedProduct : product))
  );

  this.closeForm();
}
```

Se explica por partes.

1. Comprobar que hay producto seleccionado

```ts
const selected = this.selectedProduct();
if (!selected) {
  return;
}
```

- Se lee el signal **una vez** y se guarda en una constante, `selected`.
- `selectedProduct` es de tipo `Product | undefined` (paso 17). Si fuera `undefined` no habría nada que actualizar, así que `return` sale del método sin hacer nada más.
- Después del `if`, TypeScript **sabe** que `selected` ya no puede ser `undefined` y deja usarlo como `Product`. Sin esta comprobación, al escribir `...selected` o `selected.id` TypeScript daría un error.

A esta forma de empezar un método, saliendo pronto si algo no se cumple, se le llama **cláusula de guarda** (*guard clause*). Evita meter todo el resto del método dentro de un `if`.

En la práctica, `save()` solo llama a `updateProduct()` en modo `'edit'`, y en ese modo siempre hay producto seleccionado (paso 17: los dos signals se cambian juntos). La comprobación es necesaria igualmente porque TypeScript no puede saber esa relación entre los dos signals.

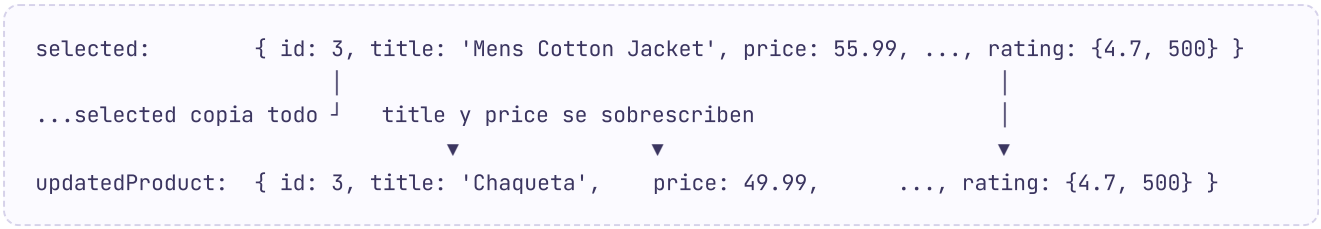
2. El spread de objetos: crear el producto actualizado

```ts
const updatedProduct: Product = {
  ...selected,
  title: this.title(),
  price: this.price() || 0,
  description: this.description(),
  category: this.category(),
  image: this.image()
};
```

En el paso 12 se usó el operador spread (`...`) con **arrays**: `[...products, product]` copia los elementos de un array en otro nuevo. Con **objetos** funciona igual, pero copia **propiedades**:

- `{ ... }` crea un **objeto nuevo**.
- `...selected` copia dentro **todas las propiedades** del producto seleccionado: `id`, `title`, `price`, `description`, `category`, `image` y `rating`.
- Las propiedades que se escriben **después** sustituyen a las copiadas. Aquí se sustituyen las cinco que se pueden editar con los valores de los campos.

El resultado es un producto nuevo que **conserva el** `id` **y el** `rating` del original (porque no se sobrescriben) y tiene los datos editados:



El orden importa. Si el spread fuera al final, copiaría los valores antiguos encima de los nuevos y no cambiaría nada:

```ts
// ❌ MAL: ...selected va al final y vuelve a poner el título antiguo
const updatedProduct: Product = {
  title: this.title(),
  ...selected
};
```

Con `price: this.price() || 0` se trata el precio vacío igual que en `addProduct()` (paso 13).

El error: modificar el objeto directamente

Podría parecer más sencillo cambiar las propiedades del producto seleccionado:

```ts
// ❌ MAL
selected.title = this.title();
selected.price = this.price();
// ...
```

Esto **muta** el objeto. Como `selected` es **el mismo objeto** que está dentro del array de `products` (no una copia), se estaría cambiando el producto "por debajo", sin pasar por `set` ni `update`:

- El signal `products` no se entera del cambio: su array sigue siendo el mismo, con el mismo objeto dentro. Si no se hace nada más, Angular no tiene por qué volver a pintar la card.
- Cualquier otro sitio que tuviera ese objeto lo vería cambiar de repente. Es justo lo que la inmutabilidad quiere evitar: que un dato que ya existe cambie "por sorpresa" (paso 12).

La regla es la misma para arrays y para objetos: **no se modifican, se crea uno nuevo con los cambios**.

3. `map`: sustituir el producto en la lista

```ts
this.products.update(products =>
  products.map(product => (product.id === updatedProduct.id ? updatedProduct : product))
);
```

`map` recorre el array y crea **un array nuevo del mismo tamaño**. Para cada elemento ejecuta la función que se le pasa, y **lo que devuelve la función es lo que se pone en esa posición** del array nuevo.

La función usa el operador ternario:

```ts
product.id === updatedProduct.id ? updatedProduct : product
```

- Si el producto es **el que se ha editado** (mismo `id`), en su lugar se pone `updatedProduct`.
- Si es **cualquier otro**, se pone el mismo producto, sin cambios.

Por ejemplo, al editar el producto con `id` 3:

```
products:  [ p1, p2, p3,             p4, ... ]
             |   |   |               |
id === 3:  false false true          false
             |   |   |               |
nuevo:     [ p1, p2, updatedProduct, p4, ... ]   ← array NUEVO, mismo tamaño
```

El producto editado **se queda en la misma posición** (no se mueve al final, como pasaba al crear una copia en el paso 17).

Comparación de `filter` y `map`:

Método	Tamaño del array nuevo	Para qué se usa en el CRUD
`filter`	igual o menor	**borrar**: quitar elementos
`map`	**siempre igual**	**editar**: transformar o sustituir

Igual que `filter`, `map` **no modifica el array original**. El signal recibe un array distinto, avisa a Angular y, gracias al `track product.id`, el `@for` solo vuelve a pintar la card del producto 3: las demás tienen el mismo objeto que antes.

Los paréntesis alrededor del ternario (`product => (product.id === ... ? ... : product)`) no son obligatorios; se ponen para que se lea claramente que todo el ternario es lo que devuelve la función flecha.

El nombre `updatedProduct` se ha elegido para no repetir `product`, que ya es el parámetro de la función flecha de `map`. Si la constante también se llamara `product`, dentro de `map` no se podría distinguir una de otra.

4. Cerrar el div

Por último, `closeForm()` vacía los campos, vuelve al modo `'new'`, quita el producto seleccionado y cierra el `div` (paso 17).

Borrar el producto que se está editando

¿Qué pasa si se pulsa **Editar** en un producto y, con el `div` abierto, se pulsa **Borrar** en ese mismo producto? El producto desaparecería de la lista, pero el `div` seguiría abierto editándolo. Al pulsar **Guardar**, `map` no encontraría ningún producto con ese `id` y no haría nada: el usuario perdería los cambios sin saber por qué.

Para evitarlo, `deleteProduct()` cierra el `div` si se borra el producto que se está editando:

```ts
// Borra el producto con el id indicado.
deleteProduct(id: number) {
  // Si se está editando el producto que se borra, se cierra el div.
  const selected = this.selectedProduct();
  if (selected && selected.id === id) {
    this.closeForm();
  }

  // Inmutabilidad: filter crea un array nuevo sin el producto borrado.
  this.products.update(products => products.filter(product => product.id !== id));
}
```

- `selected && selected.id === id`: primero se comprueba que hay producto seleccionado y, **solo si lo hay**, se compara su `id`. El operador `&&` deja de evaluar en cuanto encuentra algo falso, así que si `selected` es `undefined` nunca se intenta leer `selected.id`.
- Si se borra **otro** producto mientras se edita uno, el `div` sigue abierto: la edición no se ve afectada.

El CRUD completo

Operación	Método	Cómo se modifica el signal `products`
Create (crear)	`addProduct()`	`update(products => [...products, product])`
Read (leer)	`@for`	se lee con `products()`
Update (editar)	`updateProduct()`	`update(products => products.map(...))`
Delete (borrar)	`deleteProduct()`	`update(products => products.filter(...))`

En las tres operaciones que modifican la lista, **siempre se crea un array nuevo** y, al editar, también **un objeto nuevo**.

Qué pasa al pulsar Guardar en modo edición

1. El usuario pulsa **Editar** en "Mens Cotton Jacket" (`id` 3), cambia el precio a 49.99 y pulsa **Guardar**.
2. Se ejecuta `save()`: `mode()` vale `'edit'`, así que llama a `updateProduct()`.
3. `updateProduct()` comprueba que hay producto seleccionado.
4. Crea `updatedProduct` con `{ ...selected, ... }`: `id` 3, valoración 4.7 (500) y el precio nuevo.
5. `map` crea un array nuevo con `updatedProduct` en la posición del producto 3, y `update` lo guarda en `products`.
6. El signal avisa a Angular y el `@for` actualiza solo la card 3, que muestra 49.99 €.
7. `closeForm()` cierra el `div` y deja todo en modo nuevo.

ETIQUETA DE GIT · PASOS 20–21

07_computed

signal **calculado** con `computed()`, `totalProducts`, que muestra junto al título el número de productos y se actualiza solo al crear o borrar.

```
$ git checkout 07_computed
```

20 Valores derivados: planteamiento

Hasta ahora todos los signals del componente guardan un valor que **se cambia a mano** con `set` o `update`: la lista de productos, si el `div` está visible, cada campo del formulario...

Pero hay datos que **no hace falta guardar**, porque se pueden **calcular** a partir de otros. Por ejemplo, el **número total de productos**: no es un dato nuevo, es simplemente la longitud del array `products`.

A estos datos se les llama **valores derivados**: su valor se deriva (se obtiene) de otro valor.

En este paso se muestra el total junto al título de la página:

```
Productos (20)
```

y el número se actualiza solo al crear o borrar un producto, sin escribir ningún código extra en `addProduct()` ni en `deleteProduct()`.

21 El signal calculado `totalProducts`: `computed()`

Qué es un signal `computed`

Un **signal** `computed` (signal calculado) es un signal cuyo valor **no se asigna**, sino que **se calcula** con una función a partir de otros signals.

Hay, por tanto, dos tipos de signals:

Tipo	Se crea con	Su valor...	¿Se puede modificar?
Signal **escribible**	`signal(...)`	se guarda y se cambia a mano	Sí, con `set` y `update`
Signal **calculado**	`computed(...)`	se calcula a partir de otros signals	**No**: es de solo lectura

Crear el signal `totalProducts`

En `products.ts` se importa la función `computed` junto a `signal`:

```ts
import { Component, computed, signal } from '@angular/core';
```

y se crea el signal justo debajo de `products`:

```ts
// Signal de tipo array con todos los productos del mock.
products = signal<Product[]>(PRODUCTS);

// Signal calculado (de solo lectura): número total de productos.
// Se vuelve a calcular automáticamente cada vez que cambia products.
totalProducts = computed(() => this.products().length);
```

Por partes:

- `computed(...)` crea el signal calculado.
- Recibe **una función** (aquí una función flecha, `() => ...`), que es la que **calcula el valor**. No recibe parámetros: lo que necesita lo lee de otros signals.
- `this.products()` lee el signal `products` (el array) y `.length` devuelve cuántos elementos tiene.
- Lo que devuelve la función es el valor del signal: al principio, `20`.
- No hace falta escribir el tipo: TypeScript **lo deduce** de lo que devuelve la función. Como `.length` es un número, `totalProducts` es de tipo `Signal<number>`.

Comparado con `products`:

```ts
products = signal<Product[]>(PRODUCTS);                 // recibe un valor inicial
totalProducts = computed(() => this.products().length); // recibe una función
```

`signal()` recibe **un valor**; `computed()` recibe **una función** que calcula el valor.

Las dependencias: cómo sabe cuándo recalcular

La función de `totalProducts` lee el signal `products`. Angular **apunta automáticamente** qué signals se leen dentro de la función: a esos signals se les llama **dependencias** del `computed`.

```
products ──────────► totalProducts ──────────► <h1> de la plantilla
(signal escribible)  (signal calculado)
se cambia con        se recalcula cuando
set / update         cambia products
```

No hay que declarar las dependencias en ningún sitio: basta con **leer** el signal dentro de la función. Si la función leyera también otro signal, ese pasaría a ser dependencia automáticamente.

Cuando cambia `products` (al crear, editar o borrar), Angular sabe que `totalProducts` puede haber cambiado, lo recalcula y actualiza el `<h1>`.

Se calcula solo cuando hace falta

Un `computed` es **perezoso** (*lazy*) y **guarda su resultado** (como una caché):

- La función **no se ejecuta al crear** el signal, sino la **primera vez que se lee**.
- Si se vuelve a leer y sus dependencias **no han cambiado**, devuelve el valor guardado **sin volver a ejecutar la función**.
- Cuando cambia una dependencia, la **siguiente lectura** vuelve a ejecutar la función.

Comprobado con un contador de ejecuciones (con una lista de 3 elementos):

Acción	Valor leído	Veces que se ha ejecutado la función
Crear el `computed`	—	0
Leer `totalProducts()`	3	1
Leerlo otra vez	3	1 (usa el valor guardado)
`products.update(...)` con un array nuevo	—	1 (todavía no se ha leído)
Leer `totalProducts()`	4	2

Por eso se puede leer `totalProducts()` en la plantilla todas las veces que se quiera: los productos solo se cuentan de nuevo cuando la lista cambia.

Es de solo lectura: no se puede modificar

Un signal `computed` **no tiene** `set` **ni** `update`. Su valor depende siempre de la función; no se le puede asignar otro:

```ts
// ❌ MAL: no compila
this.totalProducts.set(5);
this.totalProducts.update(total => total + 1);
```

El compilador da estos errores:

```
TS2339: Property 'set' does not exist on type 'Signal<number>'.
TS2339: Property 'update' does not exist on type 'Signal<number>'.
```

Fíjate en el tipo: `products` es un `WritableSignal<Product[]>` (signal **escribible**), pero `totalProducts` es un `Signal<number>`, que **solo permite leer** con `totalProducts()`.

Si se quiere que cambie el total, hay que cambiar **el dato del que depende**:

```ts
// ✅ BIEN: se modifica products y totalProducts se recalcula solo
this.products.update(products => [...products, product]);
```

Es lo que ya hacen `addProduct()` y `deleteProduct()`, por eso **no ha hecho falta tocarlos**.

Que sea de solo lectura es una ventaja: el total **nunca puede quedar desincronizado** con la lista. Si fuera un signal escribible, habría que acordarse de actualizarlo en cada método que añade o borra productos:

```ts
// ❌ Sin computed: un signal más que hay que mantener a mano
totalProducts = signal(PRODUCTS.length);

addProduct() {
  // ...
  this.products.update(products => [...products, product]);
  this.totalProducts.update(total => total + 1); // si se olvida, el total es incorrecto
}

deleteProduct(id: number) {
  // ...
  this.products.update(products => products.filter(product => product.id !== id));
  this.totalProducts.update(total => total - 1); // y aquí también
}
```

```ts
// ✅ Con computed: una línea, y siempre correcto
totalProducts = computed(() => this.products().length);
```

`computed` también necesita la inmutabilidad

Un `computed` se recalcula **cuando su dependencia avisa de que ha cambiado**. Como se vio en el paso 12, si se modifica el array con `push`, el signal `products` no avisa, así que `totalProducts` **tampoco se recalcula**:

```ts
// ❌ MAL
this.products.update(products => {
  products.push(product);
  return products;
});
// El array tiene un elemento más, pero totalProducts() sigue dando el valor anterior
```

Comprobado: tras el `push`, el array tenía 5 elementos y `totalProducts()` seguía devolviendo 4. Con `[...products, product]` se crea un array nuevo, `products` avisa y el total se actualiza.

Mostrar el total en la plantilla

En `products.html` se interpola el signal en el título:

```html
<h1>Productos ({{ totalProducts() }})</h1>
```

- Se lee **igual que cualquier signal**: llamándolo con paréntesis, `totalProducts()`. Desde la plantilla no se nota si un signal es escribible o calculado.

- Los paréntesis de fuera de las llaves, `( ... )`, son texto normal que se muestra en la página: `Productos (20)`.

Qué pasa al crear, editar o borrar un producto

1. Al cargar la página, la plantilla lee `totalProducts()`. La función se ejecuta por primera vez y devuelve `20`: se ve **Productos (20)**.
2. El usuario borra un producto: `deleteProduct()` hace `products.update(...)` con `filter`, que crea un array nuevo de 19 productos.
3. `products` avisa de que ha cambiado, y Angular sabe que `totalProducts` depende de él.
4. Angular vuelve a pintar la plantilla, lee `totalProducts()`, la función se ejecuta otra vez y devuelve `19`: se ve **Productos (19)**.
5. Al crear un producto pasa lo mismo con `[...products, product]`: **Productos (20)**.
6. Al **editar** un producto, `map` también crea un array nuevo, así que `totalProducts` se recalcula, pero da el mismo número: la longitud no cambia.

Resumen

- `computed(() => ...)` crea un signal cuyo valor se **calcula** a partir de otros signals: un **valor derivado**.
- Sus **dependencias** son los signals que se leen dentro de la función; Angular las detecta solo.
- Solo se recalcula cuando ha cambiado una dependencia y se vuelve a leer; si no, devuelve el valor guardado.
- Es de **solo lectura**: no tiene `set` ni `update`. Para cambiar su valor, se cambia el signal del que depende.
- Se lee igual que cualquier signal: `totalProducts()`.

ETIQUETA DE GIT · PASOS 22–23

08_filtro_titulo

filtro por título con un signal calculado, `filteredProducts`, que depende de dos signals (`products` y `searchText`), y un `computed` que depende de otro `computed` para contar los productos filtrados.

```
$ git checkout 08_filtro_titulo
```

22 Filtrar productos: planteamiento

Con 20 productos ya cuesta encontrar uno concreto. En este paso se añade un **filtro por título**: un campo de texto, a la izquierda del botón **Nuevo**, en el que al escribir solo se muestran las cards cuyo título contiene ese texto.

```
Productos (4 de 20)

Buscar por título [ jacket        ]  [Nuevo]

[Mens Cotton Jacket] [Snowboard Jacket] [Biker Jacket] [Rain Jacket]
```

Es un ejemplo más avanzado de `computed()` que el del paso 21:

Paso 21: `totalProducts`	**Paso 23:** `filteredProducts`
Depende de **un** signal (`products`)	Depende de **dos** signals (`products` y `searchText`)
Calcula un número (`.length`)	Calcula un **array** nuevo (con `filter`)
Función de una línea	Función con varias líneas (`{ ... }` y `return`)
Depende de un signal escribible	Otro `computed` (`totalFilteredProducts`) depende de él

El título mostrará **dos totales**: cuántos productos se ven y cuántos hay en total, por ejemplo **Productos (4 de 20)**.

23 El filtro: `searchText`, `filteredProducts` y `totalFilteredProducts`

Los signals nuevos

En este paso se añaden tres signals:

Signal	Tipo	Se crea con	Para qué sirve
`searchText`	escribible (`string`)	`signal()`	Guarda el texto que se escribe en el filtro
`filteredProducts`	calculado (`Product[]`)	`computed()`	Los productos cuyo título contiene ese texto
`totalFilteredProducts`	calculado (`number`)	`computed()`	Cuántos productos cumplen el filtro

Junto con los que ya había, quedan así:



- Las flechas indican **"se calcula a partir de"**.
- Solo hay **dos** signals escribibles: `products` (los datos) y `searchText` (lo que escribe el usuario). Todo lo demás se **calcula** a partir de ellos.

1. El signal `searchText` : el texto del filtro

En `products.ts` :

```ts
// Texto escrito en el filtro.
searchText = signal('');
```

Es un signal **escribible**, igual que los de los campos del paso 10 (`title` , `description` ...):

- Su valor inicial es `''` , la **cadena vacía**: al cargar la página el filtro no tiene nada escrito.
- TypeScript deduce que es de tipo `string` a partir del valor inicial.

Para guardar en él lo que escribe el usuario se usa un método, como en el paso 11:

```ts
// Guarda en searchText lo que se escribe en el filtro.
setSearchText(event: Event) {
  const input = event.target as HTMLInputElement;
  this.searchText.set(input.value);
}
```

Y en `products.html` , a la izquierda del botón **Nuevo**:

```HTML
<search>
  <label for="search">Buscar por título</label>
  <input id="search" type="search" [value]="searchText()" (input)="setSearchText($event)" />
</search>

<button type="button" (click)="newProduct()" [attr.aria-
expanded]="showForm()">Nuevo</button>
```

- `[value]="searchText()"` y `(input)="setSearchText($event)"`: es el mismo enlace en los dos sentidos que en los campos del paso 11. Cada tecla que se pulsa ejecuta `setSearchText()`, que guarda el texto en el signal.
- `<search>` es una etiqueta de HTML para agrupar un **buscador o filtro**. No cambia nada visualmente, pero indica a los lectores de pantalla que ahí hay una zona de búsqueda. Además permite darle estilos sin usar clases.
- `<label for="search">` con `id="search"`: como en el resto de campos, la etiqueta queda asociada al campo. Es necesaria para la accesibilidad: un lector de pantalla anuncia "Buscar por título" al llegar al campo.
- `type="search"` es un campo de texto pensado para búsquedas. En navegadores como Chrome muestra una **×** para borrar el texto, y al pulsarla también se lanza el evento `input`, así que el filtro se vacía.

Hasta aquí, `searchText` solo **guarda** el texto: todavía no filtra nada.

2. Por qué hace falta un array nuevo: `filteredProducts`

Para mostrar solo algunos productos, la primera idea podría ser **filtrar directamente** `products`. Hay que ver por qué no funciona.

❌ Error 1: filtrar el signal `products`

```TS
// ❌ MAL
setSearchText(event: Event) {
  const input = event.target as HTMLInputElement;
  this.searchText.set(input.value);
  this.products.update(products =>
    products.filter(product =>
product.title.toLowerCase().includes(input.value.toLowerCase()))
  );
}
```

`filter` crea un array nuevo **sin** los productos que no cumplen el filtro, y `update` lo guarda en `products`. Los productos que no coinciden **se pierden**:

```
products al principio:      [ 20 productos ]
se escribe "jacket":        [ 4 productos ]    ← los otros 16 ya no están
se borra el texto:          [ 4 productos ]    ← no vuelven: no hay de dónde sacarlos
se escribe "gold":          [ ]                ← se filtran los 4 que quedaban
```

Filtrar **no es borrar**. `products` es la **fuente de la verdad**: la lista real de productos. El filtro solo cambia **qué se ve**, no qué productos existen.

❌ Error 2: un segundo signal escribible

Otra idea sería tener un segundo signal, `filteredProducts`, y guardar en él el resultado de filtrar:

```ts
// ❌ MAL: un signal escribible que hay que mantener a mano
filteredProducts = signal<Product[]>(PRODUCTS);

setSearchText(event: Event) {
  // ...
  this.filteredProducts.set(/* products filtrado */);
}
```

Esto ya no pierde productos, pero ahora hay **dos listas** que tienen que coincidir, y habría que volver a filtrar **en cada método que cambia** `products`:

Método	Cambia `products`	Habría que actualizar `filteredProducts`
`setSearchText()`	no	sí (cambia el texto)
`addProduct()`	sí	sí
`updateProduct()`	sí	sí
`deleteProduct()`	sí	sí

Si se olvida en uno, la lista que se ve deja de coincidir con la real: por ejemplo, se crea un producto y no aparece, o se borra y su card sigue ahí. Es el mismo problema que con `totalProducts` en el paso 21, pero mayor.

✅ La solución: un signal `computed`

`filteredProducts` es un **valor derivado**: se puede calcular a partir de `products` y de `searchText`. Por eso es un `computed`:

- `products` no se toca nunca al filtrar: los 20 productos siguen ahí.
- `filteredProducts` es un **array nuevo**, distinto de `products`, con solo los productos que coinciden.
- Angular lo recalcula solo cuando cambia **cualquiera** de los dos signals, así que nunca queda desincronizado. No hay que tocar `addProduct()`, `updateProduct()` ni `deleteProduct()`.

```
products  ──► [ 20 productos ]   ← la lista real, no cambia al filtrar
                    │
                    │ filter (título contiene searchText)
                    ▼
filteredProducts ──► [ 4 productos ]   ← array nuevo, solo para mostrar
```

3. El signal `filteredProducts`, línea a línea

```ts
// Signal calculado: productos cuyo título contiene el texto del filtro,
// sin distinguir mayúsculas y minúsculas. Se vuelve a calcular cuando
// cambia products o cuando cambia searchText. Con el filtro vacío,
// includes('') devuelve true y se obtienen todos los productos.
filteredProducts = computed(() => {
  const text = this.searchText().trim().toLowerCase();
  return this.products().filter(product => product.title.toLowerCase().includes(text));
});
```

Una función con llaves y `return`

En el paso 21 la función era de una sola expresión, sin llaves:

```ts
totalProducts = computed(() => this.products().length);
```

Aquí hacen falta **dos instrucciones** (preparar el texto y filtrar), así que la función flecha lleva **llaves** `{ }` y hay que escribir `return` para indicar qué valor devuelve. Ese valor es el del signal.

Forma	Ejemplo	Devuelve
Sin llaves (una expresión)	`() => this.products().length`	la expresión, automáticamente
Con llaves (varias líneas)	`() => { ...; return x; }`	lo que indique `return`

Si con llaves se olvida el `return`, la función devuelve `undefined`.

Primera línea: preparar el texto

```ts
const text = this.searchText().trim().toLowerCase();
```

Se encadenan tres operaciones, de izquierda a derecha:

1. `this.searchText()` lee el signal: el texto tal y como lo ha escrito el usuario. Al leerlo, `searchText` pasa a ser **dependencia** del `computed`.
2. `.trim()` quita los **espacios del principio y del final**. Así, si se escribe `" jacket "`, se busca `"jacket"`.
3. `.toLowerCase()` lo pasa todo a **minúsculas**.

Lo escrito	Tras `trim()`	Tras `toLowerCase()` (`text`)
`''`	`''`	`''`
`'Jacket'`	`'Jacket'`	`'jacket'`
`' JaCKet '`	`'JaCKet'`	`'jacket'`
`' '`	`''`	`''`

Se guarda en una constante, `text`, para no repetir la operación con cada producto.

Segunda línea: filtrar los productos

```ts
return this.products().filter(product => product.title.toLowerCase().includes(text));
```

1. `this.products()` lee el signal con todos los productos. Al leerlo, `products` pasa a ser **la otra dependencia** del `computed`.
2. `.filter(...)` recorre el array y crea **un array nuevo** solo con los productos para los que la función devuelve `true` (como en el paso 15). **No modifica** `products`.
3. Para cada `product`:
   - `product.title.toLowerCase()` pasa el título a minúsculas.
   - `.includes(text)` devuelve `true` si el título **contiene** `text` en cualquier posición, y `false` si no.

`includes` es un método de los textos (`string`) que indica si un texto **contiene** a otro:

```ts
'mens cotton jacket'.includes('jacket');  // true  (está al final)
'mens cotton jacket'.includes('cotton');  // true  (está en medio)
'mens cotton jacket'.includes('gold');    // false (no está)
```

Por qué se pasan los dos textos a minúsculas

`includes` **distingue mayúsculas y minúsculas**:

```ts
'Mens Cotton Jacket'.includes('cotton');                // false: "Cotton" ≠ "cotton"
'Mens Cotton Jacket'.toLowerCase().includes('cotton');  // true
```

Al pasar a minúsculas **tanto el título como el texto buscado**, da igual cómo escriba el usuario: `jacket`, `Jacket` o `JACKET` encuentran los mismos productos. Si solo se pasara uno de los dos, `JACKET` no encontraría nada.

4. La primera vez: `includes('')` y la lista completa

Una duda importante: **¿cómo consigue `filteredProducts` tener todos los productos al cargar la página**, si no se ha escrito nada en el filtro? No se inicializa con `PRODUCTS` ni hay ningún `if` del tipo "si el filtro está vacío, devuelve todos".

La respuesta está en cómo funciona `includes` con la **cadena vacía**:

```ts
'mens cotton jacket'.includes('');  // true
'gold ring'.includes('');           // true
''.includes('');                    // true
```

Cualquier texto contiene la cadena vacía, así que `includes('')` devuelve **siempre** `true`. Es lógico: buscar "nada" dentro de un texto siempre tiene éxito.

Paso a paso, lo que pasa al cargar la página:

1. `searchText` se crea con su valor inicial, `''`.
2. `filteredProducts` se crea, pero **todavía no se calcula**: un `computed` es perezoso (paso 21).
3. Angular pinta la plantilla y el `@for` lee `filteredProducts()`. Es la **primera lectura**, así que se ejecuta la función:
   - `text` vale `''.trim().toLowerCase()`, es decir, `''`.
   - `filter` recorre los 20 productos y, para cada uno, `título.includes('')` devuelve `true`.
   - Como **todos** dan `true`, `filter` devuelve un array nuevo con **los 20 productos**.
4. El `@for` pinta las 20 cards y el título muestra **Productos (20 de 20)**.

Por tanto, **la cadena vacía hace de "sin filtro"** sin escribir ningún código especial. Lo mismo pasa cuando el usuario **borra** lo que había escrito, o si solo escribe **espacios**: `trim()` los quita, `text` vuelve a ser `''` y se ven otra vez todos los productos.

Comprobado: con el filtro vacío, `filteredProducts()` devuelve todos los productos, pero **en un array distinto** de `products()` (la comparación `filteredProducts() === products()` da `false`), porque `filter` siempre crea un array nuevo.

5. Dos dependencias: cuándo se recalcula

Dentro de la función se leen **dos** signals, así que `filteredProducts` tiene dos dependencias y se recalcula cuando cambia **cualquiera** de ellas:

Qué ocurre	Signal que cambia	`filteredProducts` se recalcula
El usuario escribe en el filtro	`searchText`	sí
Se crea un producto (Guardar)	`products`	sí
Se edita un producto (Guardar)	`products`	sí
Se borra un producto	`products`	sí
Se abre o cierra el `div` de alta	`showForm`	no: no es dependencia
Se escribe en un campo del `div`	`title`, `price` …	no: no es dependencia

Comprobado con un contador de ejecuciones sobre una lista de prueba con 3 productos (`Mens Cotton Jacket`, `Gold Ring` y `Rain Jacket`):

Acción	`filteredProducts()`	Ejecuciones
Crear el `computed`	—	0
Primera lectura (filtro vacío)	los 3	1
`searchText` pasa a `'JACKET'`	`Mens Cotton Jacket`, `Rain Jacket`	2
Se añade `Denim Jacket`	los 2 anteriores y `Denim Jacket`	3
Se borra `Mens Cotton Jacket`	`Rain Jacket`, `Denim Jacket`	4

Fíjate en la cuarta fila: al **crear** un producto que coincide con el filtro, aparece directamente en la lista filtrada sin haber tocado `addProduct()`.

6. Un `computed` que depende de otro: `totalFilteredProducts`

```ts
// Signal calculado: número de productos que cumplen el filtro.
totalFilteredProducts = computed(() => this.filteredProducts().length);
```

Es igual que `totalProducts`, pero cuenta los productos de `filteredProducts` en vez de los de `products`.

Lo nuevo es que su dependencia, `filteredProducts`, **es otro** `computed`. Un `computed` se puede leer dentro de otro exactamente igual que un signal escribible, y se forma una **cadena**:



Cuando cambia `searchText`, se recalcula `filteredProducts` y, como ha cambiado, también `totalFilteredProducts`. No hay que hacer nada para encadenarlos: Angular sigue las dependencias solo.

Así quedan los dos totales:

Signal	Cuenta	Depende de	Con "jacket"
`totalProducts`	todos los productos	`products`	20
`totalFilteredProducts`	los productos que cumplen filtro	`filteredProducts`	4

`totalProducts` **no cambia al filtrar**, porque `products` no cambia. Solo cambia al crear o borrar.

Igual que los demás `computed`, los tres signals calculados son de **solo lectura**: no tienen `set` ni `update` (paso 21). Para cambiar lo que se muestra hay que cambiar `searchText` o `products`.

7. La plantilla

El título con los dos totales

```html
<h1>Productos ({{ totalFilteredProducts() }} de {{ totalProducts() }})</h1>
```

Dos interpolaciones en el mismo texto. Al cargar la página se ve **Productos (20 de 20)**; al escribir "jacket", **Productos (4 de 20)**.

El `@for` recorre `filteredProducts()`

```html
@for (product of filteredProducts(); track product.id) {
```

Es el **único cambio** necesario en el listado: antes recorría `products()` y ahora recorre `filteredProducts()`. Las cards, los botones **Editar** y **Borrar** y el `track product.id` siguen igual.

Gracias a `track product.id`, al filtrar Angular **no vuelve a crear** las cards que siguen visibles: solo quita las que dejan de cumplir el filtro y vuelve a crear las que lo cumplen de nuevo.

El mensaje de `@empty`

```HTML
} @empty {
  <p>No se han encontrado productos.</p>
}
```

`@empty` se muestra cuando el array que recorre el `@for` está vacío. Ahora ese array es `filteredProducts()`, que puede quedar vacío por **dos motivos**:

- no hay ningún producto (se han borrado todos), o
- hay productos, pero **ninguno cumple el filtro** (por ejemplo, al escribir "xyz").

Por eso el mensaje cambia de "No hay productos." a **"No se han encontrado productos."**, que sirve para los dos casos.

8. Editar y borrar con el filtro activo

Los botones **Editar** y **Borrar** de las cards no cambian, y siguen funcionando con el filtro activo porque trabajan **sobre** `products`, buscando el producto por su `id`, no sobre la lista filtrada:

- **Borrar**: `deleteProduct()` quita el producto de `products` con `filter`. `filteredProducts` se recalcula y la card desaparece. Los dos totales bajan en 1.
- **Editar**: `updateProduct()` sustituye el producto en `products` con `map`. `filteredProducts` se recalcula con el título nuevo. Si con el cambio el título **deja de contener** el texto del filtro, la card **desaparece** de la lista al guardar: el producto sigue existiendo, pero ya no cumple el filtro. Al borrar el filtro vuelve a verse.
- **Crear**: si el producto nuevo no cumple el filtro, tampoco se ve hasta borrar el filtro, aunque `totalProducts` sí aumenta.

Este es el beneficio de tener **una sola fuente de la verdad** (`products`) y todo lo demás calculado: cada operación modifica un único signal y todo lo que depende de él se actualiza solo.

9. Estilos

`<search>` es un elemento de bloque, así que por defecto ocuparía una línea entera y el botón **Nuevo** quedaría debajo. En `products.css`:

```css
/* Filtro: en la misma línea que el botón Nuevo, a su izquierda */
search {
  display: inline-flex;
  align-items: center;
  gap: 0.5rem;
  margin-right: 0.5rem;
}

search > label {
  margin-top: 0;
}
```

- `display: inline-flex`: el `<search>` se comporta como un elemento **en línea**, así que el botón **Nuevo** queda a su derecha. Por dentro es una fila *flex*: la etiqueta y el campo, uno al lado del otro.
- `align-items: center`: centra verticalmente la etiqueta y el campo.
- `gap` y `margin-right`: separación entre etiqueta, campo y botón.
- `search > label`: la regla `label` de `products.css` (añadida con los campos del producto en `03_alta_producto`) da a todas las etiquetas un margen superior para separarlas del campo anterior en el `div` de alta. La del filtro no lo necesita porque está en la misma fila que su campo, así que se anula con `margin-top: 0`.

Qué pasa al escribir en el filtro

1. Al cargar la página, `searchText` vale `''`. El `@for` lee `filteredProducts()` y, como `includes('')` siempre es `true`, se ven los 20 productos: **Productos (20 de 20)**.
2. El usuario escribe `j` en el filtro. El evento `input` llama a `setSearchText($event)`, que guarda `'j'` en `searchText`.
3. `searchText` avisa de que ha cambiado. Angular sabe que `filteredProducts` depende de él, y que `totalFilteredProducts` depende de `filteredProducts`.
4. Angular vuelve a pintar: lee `filteredProducts()`, la función se ejecuta con `text = 'j'` y devuelve los productos cuyo título contiene una "j".
5. El usuario sigue escribiendo hasta `jacket`. Con cada tecla se repiten los pasos 2 a 4. Al final quedan **4 productos** (Mens Cotton Jacket, el de Snowboard, el de Biker y el Rain Jacket): **Productos (4 de 20)**.
6. El usuario borra el texto: `searchText` vuelve a `''`, `includes('')` da `true` para todos y se ven otra vez los 20.
7. Durante todo el proceso, `products` **no ha cambiado**: sigue teniendo los 20 productos.

Resumen

- `searchText` es un signal **escribible** con el texto del filtro; se modifica con `setSearchText()`, igual que los campos del paso 11.
- `filteredProducts` es un signal **calculado** con los productos cuyo título contiene ese texto. Es un **array nuevo**: `products` no se modifica nunca al filtrar, porque filtrar no es borrar.

- Tiene **dos dependencias**, `products` y `searchText`, y se recalcula cuando cambia cualquiera de las dos.
- `toLowerCase()` en los dos textos hace que el filtro no distinga mayúsculas y minúsculas, y `trim()` ignora los espacios de los extremos.
- `includes('')` **siempre devuelve** `true`: con el filtro vacío se obtienen todos los productos, también la primera vez que se calcula, sin inicializar nada ni escribir ningún `if`.
- `totalFilteredProducts` es un `computed` que **depende de otro** `computed`; Angular sigue la cadena de dependencias solo.
- El `@for` recorre `filteredProducts()` y el `@empty` muestra un único mensaje para la lista vacía y para el filtro sin resultados.

ETIQUETA DE GIT · PASOS 24–25

09_filtro_categoria

filtro por categoría con un `<select>` y el signal `searchCategory`, que se combina con el filtro por título dentro del mismo `filteredProducts`.

```
$ git checkout 09_filtro_categoria
```

24 Filtrar por categoría: planteamiento

Además de buscar por título, en este paso se podrá **filtrar por categoría**. A la derecha del filtro por título se añade un desplegable (`<select>`) con las cuatro categorías de los productos y una opción más, **Todas**, para no filtrar por categoría:

```
Productos (3 de 20)

Buscar por título [ jacket ]  Categoría [ women's clothing ▾ ]  [Nuevo]
                                        ┌────────────────────┐
                                        │ Todas              │
                                        │ men's clothing     │
                                        │ women's clothing   │
                                        │ jewelery           │
                                        │ electronics        │
                                        └────────────────────┘
```

Los dos filtros **se combinan**: solo se ven los productos que cumplen **los dos a la vez**. En el ejemplo, los que tienen "jacket" en el título **y** son de la categoría *women's clothing*: 3 de los 20.

Para hacerlo:

- Un signal escribible nuevo, `searchCategory`, con la categoría elegida.
- Un método, `setSearchCategory()`, que se ejecuta con el evento `change` del `<select>` y guarda la categoría en el signal.
- **No se crea un `computed` nuevo**: se amplía `filteredProducts` para que tenga en cuenta también la categoría.

25 El filtro por categoría: `searchCategory` y `filteredProducts`

1. El signal `searchCategory`

En `products.ts`, debajo de `searchText`:

```ts
// Categoría elegida en el filtro: una del enum Category o 'all' (Todas).
searchCategory = signal<Category | 'all'>('all');
```

El tipo: `Category | 'all'`

El signal puede guardar **dos clases de valores**:

- Una de las categorías del enum `Category` (paso 1): `Category.MenClothing`, `Category.WomenClothing`, `Category.Jewelery` o `Category.Electronics`.
- El texto literal `'all'`, que significa "Todas": no filtrar por categoría.

Es un **tipo unión** como los del paso 17: el `|` se lee "o". Aquí se une un **enum** con un **literal**. TypeScript solo deja guardar esos valores:

```ts
this.searchCategory.set(Category.Jewelery); // ✅ una categoría
this.searchCategory.set('all');             // ✅ Todas
this.searchCategory.set('todas');           // ❌ error: no es Category ni 'all'
```

El error del último caso es:

```
TS2345: Argument of type '"todas"' is not assignable to parameter of type 'Category | "all"'.
```

Hace falta escribir el tipo entre `< >`. Si no, `signal('all')` sería de tipo `string` y aceptaría cualquier texto.

El valor inicial: `'all'`

Al cargar la página no se filtra por categoría, así que el valor inicial es `'all'`. Es el equivalente a la cadena vacía `''` de `searchText`: el valor que significa **"sin filtro"**.

¿Por qué `'all'` y no, por ejemplo, `''` o `undefined`? Porque **se lee mejor**: `category === 'all'` dice claramente "se han elegido todas". Además no puede confundirse con ninguna categoría real.

El nombre: `searchCategory`

El componente ya tiene un signal `category`: el campo **Categoría** del `div` de alta y edición (paso 10). Son dos cosas distintas:

Signal	Dónde está	Para qué sirve	Tipo
`category`	en el `div` de alta / edición	la categoría del producto que se crea/edita	`Category`
`searchCategory`	en el filtro, junto a **Nuevo**	la categoría por la que se filtra la lista	`Category | 'all'`

El prefijo `search` lo relaciona con `searchText`: los dos signals del filtro empiezan igual.

2. El `<select>` en la plantilla

En `products.html`, dentro del `<search>` y a continuación del filtro por título:

```HTML
<search>
  <label for="search">Buscar por título</label>
  <input id="search" type="search" [value]="searchText()" (input)="setSearchText($event)" />

  <label for="search-category">Categoría</label>
  <select id="search-category" (change)="setSearchCategory($event)">
    <option value="all" [selected]="searchCategory() === 'all'">Todas</option>
    @for (option of categories; track option) {
      <option [value]="option" [selected]="option === searchCategory()">{{ option }}
</option>
    }
  </select>
</search>
```

Va **dentro del mismo** `<search>` porque es otra parte del mismo filtro. Así los dos filtros quedan juntos, a la izquierda del botón **Nuevo**.

La etiqueta y el `id`

- `<label for="search-category">` asocia la etiqueta al desplegable, como en los demás campos. Es necesaria para la accesibilidad.
- El `id` es `search-category` y no `category` porque **ya existe** un campo con `id="category"`: el desplegable del `div` de alta (paso 11). Cuando el `div` está abierto, los dos desplegables están en la página a la vez, y **un** `id` **no se puede repetir** en una página: si se repitiera, el `<label for="category">` no sabría a cuál de los dos referirse.

La opción "Todas"

```HTML
<option value="all" [selected]="searchCategory() === 'all'">Todas</option>
```

- Va **antes y fuera del** `@for`: es una opción fija que no está en el enum `Category`, así que hay que escribirla a mano. Al ser la primera, es la que se ve al cargar la página.
- `value="all"` es un **atributo normal**, sin corchetes, porque su valor es un texto fijo que no cambia. Los corchetes (`[value]`) solo hacen falta cuando el valor sale de una variable o expresión, como en las opciones del `@for`.
- El texto que ve el usuario es **"Todas"**, pero el valor que se guarda en el signal es `'all'`. El texto visible y el valor no tienen por qué coincidir.

Las opciones de las categorías: el `@for`

```HTML
@for (option of categories; track option) {
  <option [value]="option" [selected]="option === searchCategory()">{{ option }}</option>
}
```

Es el mismo `@for` que el desplegable del `div` de alta (paso 11): recorre `categories`, el array con los valores del enum (`Object.values(Category)`), y crea un `<option>` por categoría. **Se reutiliza el mismo array**: no hay que escribir las categorías otra vez.

Opción	Texto visible	Valor (`value`)
fija	Todas	`'all'`
del `@for`	men's clothing	`"men's clothing"`
del `@for`	women's clothing	`"women's clothing"`
del `@for`	jewelery	`'jewelery'`
del `@for`	electronics	`'electronics'`

`[selected]`: del signal a la pantalla

`[selected]` marca como seleccionada la opción cuyo valor coincide con el del signal, igual que en el paso 11. Es el sentido **componente → pantalla**:

- `[selected]="searchCategory() === 'all'"` en **Todas**.
- `[selected]="option === searchCategory()"` en cada categoría.

Como el valor inicial es `'all'`, al cargar la página aparece elegida **Todas**. En este paso el signal solo cambia desde el propio desplegable, pero así el `<select>` **siempre muestra lo que vale el signal**, como el resto de campos del componente.

3. El evento `change` y `setSearchCategory()`

```HTML
<select id="search-category" (change)="setSearchCategory($event)">
```

Esta vez se usa el evento `change` en lugar de `input`.

`change` frente a `input`

Evento	En un campo de texto (`<input>`)	En un `<select>`
`input`	con **cada tecla** que cambia el texto	al elegir otra opción
`change`	solo al **terminar**: al salir del campo o pulsar Intro	al elegir otra opción

- En un **campo de texto** son muy distintos. Por eso el filtro por título usa `input`: se filtra mientras se escribe, letra a letra. Con `change`, el filtro no se aplicaría hasta salir del campo.

- En un `<select>` se producen **los dos a la vez**: al elegir una opción el valor ya está decidido, así que el navegador lanza `input` y justo después `change`. Comprobado en el navegador: al cambiar de opción con el teclado, se registran los eventos `input` y `change`, en ese orden.

`change` significa "el valor **ha cambiado** y es definitivo", que es justo lo que ocurre al elegir una opción de una lista. Por eso es el evento más habitual para un `<select>`. El desplegable del `div` de alta (paso 11) usa `input` y también funciona: en un `<select>` el resultado es el mismo.

El método `setSearchCategory()`

```ts
// Guarda en searchCategory la categoría elegida en el filtro.
setSearchCategory(event: Event) {
  const select = event.target as HTMLSelectElement;
  this.searchCategory.set(select.value as Category | 'all');
}
```

Es como `setCategory()` del paso 11:

1. `event.target` es el elemento que ha lanzado el evento: el `<select>`. Con `as HTMLSelectElement` se le indica a TypeScript que es un desplegable, para poder leer su `value`.
2. `select.value` es el `value` de la opción elegida: `'all'` o el texto de una categoría. Para TypeScript, `value` es un `string` cualquiera.
3. `as Category | 'all'` le indica a TypeScript que ese texto es uno de los valores permitidos por el signal. Sin el `as`, `set` daría error, porque no acepta cualquier `string`:

   ```
   TS2345: Argument of type 'string' is not assignable to parameter of type 'Category | "all"'.
   ```

   El `as` es seguro porque las opciones del `<select>` solo tienen esos valores: `'all'` y los del enum.

4. Ampliar `filteredProducts`: dos filtros a la vez

No hace falta un `computed` nuevo: el que ya existe se amplía para que compruebe **también** la categoría.

Antes (paso 23):

```ts
filteredProducts = computed(() => {
  const text = this.searchText().trim().toLowerCase();
  return this.products().filter(product => product.title.toLowerCase().includes(text));
});
```

Ahora:

```ts
// Signal calculado: productos que cumplen los dos filtros a la vez.
// - Título: contiene el texto del filtro, sin distinguir mayúsculas y
//   minúsculas. Con el filtro vacío, includes('') devuelve true.
// - Categoría: es la elegida, o se ha elegido 'all' (Todas).
// Se vuelve a calcular cuando cambia products, searchText o searchCategory.
filteredProducts = computed(() => {
  const text = this.searchText().trim().toLowerCase();
  const category = this.searchCategory();
  return this.products().filter(
    product =>
      product.title.toLowerCase().includes(text) &&
      (category === 'all' || product.category === category)
  );
});
```

Hay dos cambios:

1. Una línea nueva, `const category = this.searchCategory();`, que lee el signal **una vez** y lo guarda en una constante, como se hace con `text`.
2. La condición del `filter` tiene ahora **dos partes** unidas con `&&`.

La condición completa

```ts
product.title.toLowerCase().includes(text) &&
(category === 'all' || product.category === category)
```

Para cada producto, `filter` lo deja en la lista si **toda** la expresión da `true`:

Parte	Pregunta
`product.title.toLowerCase().includes(text)`	¿el título contiene el texto? (filtro del paso 23)
`&&`	**y además**...
`(category === 'all' || product.category === category)`	¿la categoría vale? (filtro nuevo)

`&&` (Y lógico) da `true` solo si **las dos partes** son `true`. Así se combinan los filtros: un producto se ve si cumple el del título **y** el de la categoría.

La parte de la categoría: `category === 'all' || product.category === category`

`||` (O lógico) da `true` si **al menos una** de sus partes es `true`:

- `category === 'all'`: si se ha elegido **Todas**, esta parte es `true` para todos los productos, y **ya no hace falta mirar su categoría**: todos pasan este filtro.
- `product.category === category`: si se ha elegido una categoría concreta, la primera parte es `false` y se mira esta: el producto pasa solo si su categoría es **exactamente** la elegida.

Tabla de verdad con dos productos de ejemplo:

`category` elegida	Producto	`===` `'all'`	`product.category === category`	Resultado (`||`)
`'all'`	Mens Cotton Jacket (men's)	`true`	(no se evalúa)	`true` ✅
`'all'`	Solid Gold Petite (jewelery)	`true`	(no se evalúa)	`true` ✅
`"men's clothing"`	Mens Cotton Jacket (men's)	`false`	`true`	`true` ✅
`"men's clothing"`	Solid Gold Petite (jewelery)	`false`	`false`	`false` ❌

Igual que `&&` (paso 19), `||` **deja de evaluar en cuanto encuentra algo verdadero**: con `'all'`, la segunda comparación ni se ejecuta.

Por qué no sirve el truco de `includes('')`

En el filtro por título, el valor "sin filtro" (`''`) funciona **solo**, porque `includes('')` siempre es `true`. Con la categoría no ocurre: la comparación es con `===` (igualdad exacta), y **ninguna categoría es igual a** `'all'`:

```ts
"men's clothing" === 'all'  // false
'jewelery' === 'all'        // false
```

Si la condición fuera solo `product.category === category`, al elegir **Todas** no se vería **ningún** producto. Por eso hace falta comprobar **explícitamente** el caso `'all'` con `category === 'all' || ...`.

Filtro	Valor "sin filtro"	Cómo deja pasar todos los productos
Título	`''`	solo: `includes('')` siempre da `true`
Categoría	`'all'`	con una condición explícita: `category === 'all' || ...`

❌ El error: olvidar los paréntesis

Los paréntesis alrededor de la parte de la categoría **son imprescindibles**:

```ts
// ❌ MAL: sin paréntesis
product.title.toLowerCase().includes(text) && category === 'all' || product.category === category
```

En JavaScript, `&&` **se evalúa antes que** `||` (igual que en matemáticas se multiplica antes de sumar). Sin paréntesis, la expresión se entiende así:

```ts
(product.title.toLowerCase().includes(text) && category === 'all') || product.category === category
```

Es decir: "el título coincide **y** se han elegido todas" **o** "la categoría coincide". Con una categoría concreta, la primera parte siempre es `false`, así que **se ignora el filtro por título**: basta con que coincida la categoría.

Comprobado con los productos del mock, escribiendo `jacket` y eligiendo *jewelery*:

Condición	Productos que se ven
✅ Con paréntesis	**ninguno** (no hay joyas con "jacket" en el título)
❌ Sin paréntesis	**las 4 joyas** (Solid Gold Petite, White Gold Plated...), aunque ninguna tiene "jacket"

```ts
// ✅ BIEN: con paréntesis, primero se resuelve la parte de la categoría
product.title.toLowerCase().includes(text) &&
(category === 'all' || product.category === category)
```

5. Tres dependencias

Dentro de la función de `filteredProducts` ahora se leen **tres** signals: `searchText()`, `searchCategory()` y `products()`. Los tres son sus dependencias, y se recalcula cuando cambia **cualquiera** de ellos:



Qué ocurre	Signal que cambia	`filteredProducts` se recalcula
Se escribe en el filtro por título	`searchText`	sí
Se elige otra opción en Categoría	`searchCategory`	sí
Se crea, edita o borra un producto	`products`	sí
Se abre o cierra el `div` de alta	`showForm`	no

Lo importante es lo que **no ha hecho falta cambiar**:

- `totalFilteredProducts` sigue igual: cuenta `filteredProducts`, y `filteredProducts` ya tiene en cuenta la categoría. El título muestra el número correcto sin tocar nada.
- `totalProducts` sigue contando todos los productos: no depende de los filtros.
- El `@for` y el `@empty` siguen igual: recorren `filteredProducts()`.
- `addProduct()`, `updateProduct()` y `deleteProduct()` siguen igual.

Es la ventaja de que todo lo que se muestra se **calcule** a partir de pocos signals escribibles: para añadir un filtro basta con un signal nuevo y ampliar **un único** `computed`.

6. Ejemplos con los productos del mock

Comprobado con los 20 productos del mock:

Título	Categoría	Se ven	Título de la página
(vacío)	Todas	20	Productos (20 de 20)
(vacío)	men's clothing	4	Productos (4 de 20)
(vacío)	women's clothing	6	Productos (6 de 20)
(vacío)	jewelery	4	Productos (4 de 20)
(vacío)	electronics	6	Productos (6 de 20)
`jacket`	Todas	4	Productos (4 de 20)
`jacket`	men's clothing	1	Productos (1 de 20)
`jacket`	women's clothing	3	Productos (3 de 20)
`jacket`	jewelery	0	Productos (0 de 20) y "No se han encontrado productos."

Con el título vacío, `includes('')` da `true` para todos, así que **solo actúa el filtro por categoría**. Con **Todas**, solo actúa el filtro por título. Y con los dos vacíos ("sin filtro"), se ven los 20.

Con *jewelery* y `jacket` no hay resultados: el `@empty` muestra el mismo mensaje que en el paso 23, que sirve también para este caso.

7. Estilos

El `<select>` toma los estilos que ya tenían todos los campos (`input, select, textarea`), y su etiqueta, la regla `search > label` del paso 23 (sin margen superior). Solo se añade una regla:

```css
/* Separa el filtro por título del filtro por categoría */
search > input {
  margin-right: 0.75rem;
}
```

Dentro del `<search>`, el `gap` de `0.5rem` deja la etiqueta **Categoría** demasiado pegada al campo de texto anterior, y parecería que pertenece a él. Con un margen derecho en el campo de texto, los dos filtros quedan visualmente separados.

Qué pasa al elegir una categoría

1. Al cargar la página, `searchCategory` vale `'all'` y el `<select>` muestra **Todas** (por el `[selected]`). `filteredProducts` se calcula con la parte de la categoría siempre a `true` : se ven los 20 productos.
2. El usuario escribe `jacket` : se ven 4 productos, **Productos (4 de 20)** (paso 23).
3. El usuario elige *women's clothing* en el desplegable. El `<select>` lanza el evento `change` y se ejecuta `setSearchCategory($event)` , que guarda `"women's clothing"` en `searchCategory` .

4. `searchCategory` avisa de que ha cambiado. Angular sabe que `filteredProducts` depende de él (y `totalFilteredProducts` de `filteredProducts`).
5. Angular vuelve a pintar: `filteredProducts` se recalcula. Para cada producto se comprueba que el título contiene `jacket` **y** que la categoría es *women's clothing*. Quedan 3: **Productos (3 de 20)**.
6. El usuario elige *jewelery*: ninguna joya tiene "jacket" en el título, se ve **Productos (0 de 20)** y el mensaje "No se han encontrado productos."
7. El usuario vuelve a elegir **Todas**: `searchCategory` vuelve a `'all'` y se ven otra vez las 4 chaquetas.
8. En ningún momento ha cambiado `products`: sigue teniendo los 20.

Resumen

- `searchCategory` es un signal **escribible** de tipo `Category | 'all'`: una categoría del enum o `'all'` (Todas), que es su valor inicial y significa "sin filtro".
- El `<select>` tiene una opción fija, **Todas** (`value="all"`), y una opción por categoría creada con `@for` sobre el mismo array `categories` del `div` de alta. Su `id` es `search-category` porque `category` ya existe.
- El evento `change` se produce al elegir una opción y llama a `setSearchCategory()`, que guarda el valor con `as Category | 'all'`. En un `<select>`, `input` y `change` se producen a la vez; en un campo de texto, `change` solo al terminar de escribir.
- `filteredProducts` se amplía en lugar de crear otro `computed`: ahora tiene **tres dependencias** y combina los dos filtros con `&&`.
- `category === 'all' || product.category === category` deja pasar todos los productos con **Todas** y, si no, solo los de la categoría elegida. Al contrario que `includes('')`, la comparación con `===` necesita tratar `'all'` de forma **explícita**.
- Los **paréntesis** son imprescindibles: `&&` se evalúa antes que `||`, y sin ellos se ignoraría el filtro por título.
- `totalFilteredProducts`, `totalProducts`, el `@for`, el `@empty` y los métodos del CRUD **no cambian**: todo se actualiza solo a partir de los signals.

ETIQUETA DE GIT · PASOS 26–27

10_effect_localstorage

persistencia de los productos en `localStorage` con un `effect()`, que guarda la lista cada vez que cambia el signal `products`.

```
$ git checkout 10_effect_localstorage
```

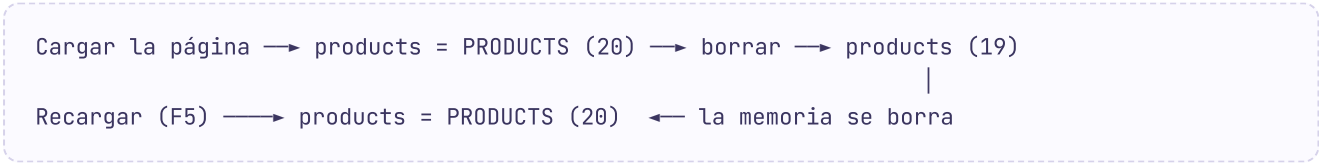
26 Persistencia de datos: planteamiento

El problema: al recargar se pierde todo

Hasta ahora, todo lo que se hace en la aplicación (crear, editar o borrar productos) **se pierde al recargar la página**. Prueba:

1. Borra un producto: se ve **Productos (19 de 19)**.
2. Pulsa F5 para recargar.
3. Vuelve a haber **20 productos**: el borrado ha desaparecido.

¿Por qué? Porque el signal `products` vive en la **memoria** del navegador, dentro del componente. Al recargar la página, el componente se crea de nuevo, y el signal vuelve a empezar con su valor inicial: el array `PRODUCTS` del mock.



Qué es la persistencia

Persistencia significa que los datos **se conservan** aunque se cierre o se recargue la aplicación: se guardan en algún sitio que **no se borra** cuando la página desaparece de la memoria.

Hay dos formas habituales de conseguirlo en una aplicación web:

Dónde se guardan	Cómo	Quién ve los datos
Base de datos en un servidor	El navegador los envía a una API (un *backend*) que los guarda en una BD (MySQL, PostgreSQL, MongoDB…)	**todos** los usuarios, desde cualquier dispositivo
Almacenamiento del navegador	El propio navegador los guarda en el disco del ordenador	**solo ese navegador**, en ese ordenador

La solución "real" de una aplicación profesional es la base de datos, pero necesita programar un servidor. Mientras tanto, el navegador ofrece un almacenamiento propio, `localStorage`, que permite **persistir datos sin base de datos ni servidor**. Es lo que se usa en este paso.

Lo que se va a hacer

- **Guardar** la lista de productos en `localStorage` **cada vez que cambie** el signal `products`. Para eso se usa un concepto nuevo de los signals: `effect()`.
- **Cargar** la lista guardada al abrir la aplicación. Si todavía no hay nada guardado (la primera vez), se usan los productos del mock.

Así, al recargar, la aplicación muestra los productos **tal como se dejaron**.

27 Guardar los productos en `localStorage` con `effect()`

1. Los signals funcionan de forma autónoma

Hasta ahora se han visto dos tipos de signals:

- Los **escribibles** (`signal()`), que **guardan** un valor y avisan cuando cambia.
- Los **calculados** (`computed()`), que **calculan** un valor a partir de otros signals y se actualizan solos.

Y la plantilla, que **lee** los signals y Angular vuelve a pintar cuando cambian.

Todo esto funciona **de forma autónoma**: cuando se hace `set` o `update` sobre `products`, Angular se encarga solo de recalcular los `computed` que dependen de él y de actualizar la pantalla. No hay que escribir ningún código para que ocurra.

```
products.update(...) ──► totalProducts, filteredProducts... se recalculan
                     └─► la plantilla se vuelve a pintar
```

Pero **eso es todo lo que pasa**. Un signal **no hace nada más** por sí solo: no guarda nada en ningún sitio, no escribe en la consola, no avisa a ningún servidor. Si queremos que **ocurra algo más cuando cambie un signal** (como guardar los productos en `localStorage`), hay que decírselo a Angular, y para eso existe `effect()`.

❌ Sin `effect`: guardar a mano en cada método

Sin `effect`, habría que guardar los productos en **cada método que cambia** `products`:

```ts
// ❌ MAL: el mismo código repetido en tres sitios
addProduct() {
  // ...
  this.products.update(products => [...products, product]);
  localStorage.setItem('products', JSON.stringify(this.products()));
}

updateProduct() {
  // ...
  this.products.update(products => products.map(...));
  localStorage.setItem('products', JSON.stringify(this.products()));
}

deleteProduct(id: number) {
  // ...
  this.products.update(products => products.filter(...));
  localStorage.setItem('products', JSON.stringify(this.products()));
}
```

Es el mismo problema que se vio con `totalProducts` en el paso 21: si mañana se añade otro método que cambia `products` (por ejemplo, "borrar todos") y se olvida esa línea, los cambios **no se guardarían** y nadie se daría cuenta hasta recargar.

✅ Con `effect`: se escribe una vez

```ts
effect(() => {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(this.products()));
});
```

El `effect` **reacciona** a los cambios de `products`, venga el cambio de donde venga. Los métodos del CRUD **no se tocan**.

2. Qué es un `effect`

Un `effect` es una función que Angular **ejecuta automáticamente** cada vez que cambia alguno de los signals que lee. Se usa para producir **efectos secundarios** (*side effects*): acciones que van **más allá de calcular un valor**, como guardar datos, escribir en la consola o modificar algo fuera de la plantilla.

Se parece a `computed` en que los dos **siguen las dependencias** solos, pero su propósito es el opuesto:

	`computed()`	`effect()`
Para qué sirve	**Calcular** un valor a partir de signals	**Hacer algo** cuando cambian unos signals
Devuelve	un signal de solo lectura con el resultado	nada que se use en la plantilla
Su función	debe ser **pura**: solo calcula y devuelve	tiene **efectos secundarios**: guarda, escribe…
Cuándo se ejecuta	cuando alguien lo **lee** (perezoso)	**siempre**, al menos una vez y tras cada cambio
Ejemplo	`totalProducts`, `filteredProducts`	guardar `products` en `localStorage`

Una forma de recordarlo: `computed` **responde a una pregunta** ("¿cuántos productos hay?"); `effect` **cumple una orden** ("cada vez que cambien los productos, guárdalos").

3. Qué es `localStorage`

`localStorage` es un almacenamiento que ofrece **el propio navegador** a cada página web. Forma parte de la **Web Storage API**, un estándar de HTML5, así que está disponible en todos los navegadores sin instalar ni importar nada: es un objeto global, `localStorage`, como `console`.

Sus características:

- **Clave y valor**: funciona como un diccionario. Cada dato se guarda con un **nombre** (la clave) y se recupera con ese mismo nombre. En este ejemplo la clave es `'products'`.
- **Solo guarda textos** (`string`): tanto la clave como el valor son textos. Para guardar arrays u objetos hay que convertirlos antes a texto (se ve en el apartado 4).
- **Es persistente**: los datos **no caducan**. Siguen ahí al recargar, al cerrar la pestaña, al cerrar el navegador e incluso al apagar el ordenador. Solo desaparecen si los borra la propia página o el usuario.
- **Es por origen**: cada **origen** (protocolo + dominio + puerto) tiene su propio `localStorage`, y una página no puede leer el de otra. Ojo: `http://localhost:4200` y `http://localhost:4300` son **orígenes distintos**, así que si se arranca la aplicación en otro puerto, no verá los productos guardados.
- **Tamaño limitado**: unos **5 MB** por origen en la mayoría de navegadores. Suficiente para datos pequeños como esta lista, no para imágenes o grandes cantidades de datos.
- **Es síncrono**: cada operación se completa en el momento, sin esperar (no hay que usar promesas ni `async`).
- **No es seguro**: cualquiera con acceso al navegador puede leerlo y modificarlo desde las herramientas de desarrollo. **Nunca** se deben guardar contraseñas, tokens o datos personales.

Verlo en el navegador

En Chrome o Edge: **F12** → pestaña **Application** (Aplicación) → **Storage** → **Local storage** → `http://localhost:4200`. Aparece una tabla con las claves y sus valores. En Firefox está en la pestaña **Storage** (Almacenamiento).

Desde ahí se puede ver el texto guardado en la clave `products`, modificarlo o borrarlo.

`localStorage` frente a `sessionStorage`

La Web Storage API tiene un segundo almacenamiento, `sessionStorage`, con los mismos métodos. La diferencia es **cuánto duran los datos**:

Almacenamiento	Los datos duran…	Compartido entre pestañas
`localStorage`	**para siempre**, hasta que se borren	sí (mismo origen)
`sessionStorage`	hasta que se **cierra la pestaña**	no: cada pestaña el suyo

Para conservar los productos entre visitas se necesita `localStorage`.

4. Los métodos de `localStorage`

Método / propiedad	Qué hace	Devuelve
`setItem(clave, valor)`	**Guarda** el valor con esa clave. Si ya existía, lo **sustituye**	nada
`getItem(clave)`	**Lee** el valor guardado con esa clave	el texto, o `null` si no existe
`removeItem(clave)`	**Borra** la clave y su valor	nada
`clear()`	**Borra todo** el `localStorage` de ese origen	nada
`key(índice)`	Nombre de la clave que está en esa posición	el nombre, o `null`
`length`	**Propiedad** (sin paréntesis): cuántas claves hay	un número

Ejemplos, comprobados en el navegador:

```ts
// Guardar (crear o sustituir)
localStorage.setItem('color', 'azul');
localStorage.setItem('color', 'rojo');     // sustituye 'azul' por 'rojo'

// Leer
localStorage.getItem('color');             // 'rojo'
localStorage.getItem('noExiste');          // null

// Borrar una clave
localStorage.removeItem('color');
localStorage.getItem('color');             // null

// Número de claves y borrar todas
localStorage.length;                       // por ejemplo, 1
localStorage.clear();
localStorage.length;                       // 0
```

`getItem` devuelve `null` cuando la clave no existe. Así se sabe si es la primera vez que se abre la aplicación (todavía no se ha guardado nada).

Todo se guarda como texto

`setItem` convierte el valor a texto **antes** de guardarlo:

```ts
localStorage.setItem('n', 5);
localStorage.getItem('n');                 // '5'  ← un string, no el número 5

// ❌ MAL: guardar un objeto directamente
localStorage.setItem('o', { a: 1 });
localStorage.getItem('o');                 // '[object Object]'  ← ¡los datos se han perdido!
```

Con un número se recupera como texto (`'5'`), pero con un objeto o un array el resultado es inservible: JavaScript lo convierte en el texto `'[object Object]'` y los datos **se pierden**.

`JSON.stringify` y `JSON.parse`

Para guardar arrays y objetos se convierten a **JSON**, un formato de texto que representa datos de JavaScript:

Función	Convierte...	Ejemplo
`JSON.stringify(dato)`	un dato (array, objeto...) **a texto**	`JSON.stringify({ a: 1 })` → `'{"a":1}'`
`JSON.parse(texto)`	un texto JSON **al dato** original	`JSON.parse('{"a":1}')` → `{ a: 1 }`

```ts
// ✅ BIEN: convertir a texto al guardar y al dato original al leer
localStorage.setItem('o', JSON.stringify({ a: 1 }));
localStorage.getItem('o');                 // '{"a":1}'
JSON.parse('{"a":1}').a;                   // 1  ← vuelve a ser un objeto
```

El ciclo completo con los productos:

```
GUARDAR:  products() ──JSON.stringify──► '[{"id":1,"title":"Fjallraven..."},...]' ──setItem──► localStorage
CARGAR:   localStorage ──getItem──► '[{"id":1,...},...]' ──JSON.parse──► Product[] ──► signal products
```

5. El `effect` que guarda los productos

En `products.ts`, primero se importa `effect` junto a los demás:

```ts
import { Component, computed, effect, signal } from '@angular/core';
```

Debajo de los `import`, fuera de la clase, una constante con la clave:

```ts
// Clave con la que se guardan los productos en localStorage.
const STORAGE_KEY = 'products';
```

Se usa una constante en vez de escribir `'products'` en cada sitio para que **guardar y cargar usen siempre la misma clave**: si se escribiera `'products'` en un sitio y `'product'` en otro por error, se guardaría con una clave y se leería con otra. Con la constante, TypeScript avisa si se escribe mal su nombre. Se escribe en MAYÚSCULAS porque es un valor fijo de configuración, como `PRODUCTS`.

Y el `effect`, en el **constructor** del componente:

```ts
constructor() {
  // Effect: cada vez que cambia products, guarda la lista en localStorage.
  // Solo se dispara con products, que es el único signal que lee.
  effect(() => {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(this.products()));
  });
}
```

Por partes:

- `effect(...)` crea el effect. Recibe **una función**, igual que `computed`.
- `this.products()` lee el signal: la lista actual de productos. Al leerlo, `products` pasa a ser la **dependencia** del effect.
- `JSON.stringify(...)` convierte el array de productos a texto.
- `localStorage.setItem(STORAGE_KEY, ...)` guarda ese texto con la clave `'products'`, sustituyendo lo que hubiera.
- La función **no devuelve nada**: no calcula un valor, **hace algo**.

Dónde se define un `effect`: el contexto de inyección

Un `effect` **no se puede crear en cualquier sitio**. Hay que crearlo en un **contexto de inyección** (*injection context*): el momento en que Angular **está creando** el componente. En la práctica, esto significa:

- ✅ En el **constructor** del componente (la opción de este ejemplo).
- ✅ En la **inicialización de una propiedad** (*field initializer*), es decir, al declararla: `saveEffect = effect(() => { ... });`.

El **constructor** es un método especial de las clases que se ejecuta **una sola vez**, cuando se crea el objeto: en este caso, cuando Angular crea el componente `Products`. Es el sitio natural para preparar cosas que deben funcionar durante toda la vida del componente, como este effect. Hasta este paso el componente no tenía constructor porque no lo había necesitado.

¿Por qué este requisito? Porque Angular **asocia el effect al componente** que se está creando. Así, cuando el componente se destruye (por ejemplo, al cambiar de página), Angular **destruye también el effect** automáticamente y deja de ejecutarlo. No hay que hacer nada para "apagarlo".

❌ Si se intenta crear un effect **fuera** de ese momento, por ejemplo dentro de un método que se ejecuta al pulsar un botón:

```ts
// ❌ MAL: un método se ejecuta mucho después de crear el componente
closeForm() {
  effect(() => { ... });
  // ...
}
```

Angular lanza este error al pulsar el botón (comprobado):

```
NG0203: effect() can only be used within an injection context such as a constructor,
a factory function, a field initializer, or a function used with `runInInjectionContext`.
```

Además, aunque funcionara, se crearía **un effect nuevo cada vez** que se pulsara el botón.

Qué signals disparan el `effect`

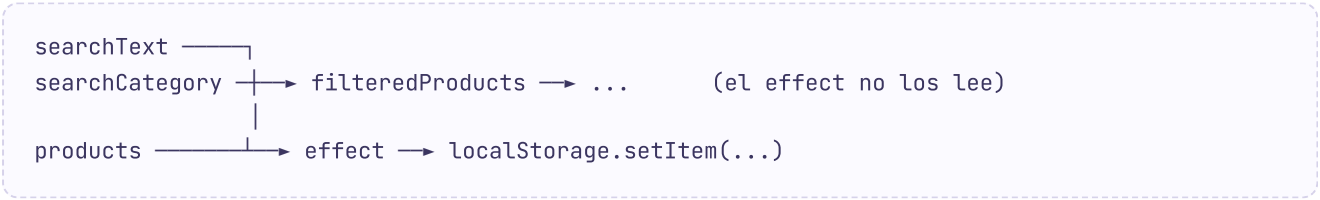
Igual que `computed`, un effect **apunta automáticamente** los signals que lee mientras se ejecuta su función: esas son sus **dependencias**, y son **los únicos signals que lo disparan** (que hacen que se vuelva a ejecutar).

Este effect solo lee `products`, así que **solo se dispara cuando cambia** `products`. Comprobado en el navegador con un `console.log` dentro del effect:

Acción del usuario	Signal que cambia	¿Se ejecuta el effect?
Abrir la página	—	**sí** (la primera vez)
Escribir en el filtro por título	`searchText`	no
Elegir una categoría en el filtro	`searchCategory`	no
Abrir el `div` con **Nuevo**	`showForm` …	no
Escribir en el campo Título del `div`	`title`	no
Guardar un producto nuevo o editado	`products`	**sí**
Borrar un producto	`products`	**sí**

Fíjate: aunque los filtros cambian lo que se ve, **no cambian** `products`, así que no se vuelve a guardar nada. Y tampoco hace falta: la lista real es la misma.

¿Y `filteredProducts`, `totalProducts`…? Tampoco lo disparan, porque el effect **no los lee**. Solo cuenta lo que se lee **dentro** de su función.



Si dentro de la función se leyeran más signals (por ejemplo, `this.searchText()`), también pasarían a dispararlo. Por eso conviene que un effect lea **solo los signals que necesita**.

Cuándo se ejecuta

- **Siempre una vez al principio**, al crearse el componente, aunque nada haya cambiado. Por eso, la primera vez que se abre la aplicación, el effect ya guarda los 20 productos del mock en `localStorage`.
- **Después de cada cambio** de sus dependencias. Pero **no en el mismo instante** del `set` o `update`: Angular lo programa y lo ejecuta poco después, al actualizar la pantalla. Si `products` cambia **varias veces seguidas** en el mismo método, el effect se ejecuta **una sola vez**, con el valor final. Comprobado: tres `update` seguidos sobre `products` produjeron **una única** ejecución del effect.

6. Cargar los productos al abrir la aplicación: `loadProducts()`

Guardar no basta: al abrir la aplicación hay que **leer** lo guardado y usarlo como valor inicial de `products`. Antes:

```ts
// Signal de tipo array con todos los productos del mock.
products = signal<Product[]>(PRODUCTS);
```

Ahora:

```ts
// Signal de tipo array con todos los productos. Su valor inicial son los
// productos guardados en localStorage o, si no hay, los del mock.
products = signal<Product[]>(this.loadProducts());
```

Y el método:

```ts
// Lee los productos guardados en localStorage. Si no hay ninguno guardado
// (la primera vez que se abre la aplicación), devuelve los del mock.
loadProducts(): Product[] {
  const saved = localStorage.getItem(STORAGE_KEY);
  if (saved === null) {
    return PRODUCTS;
  }
  return JSON.parse(saved) as Product[];
}
```

Paso a paso:

1. `localStorage.getItem(STORAGE_KEY)` lee el texto guardado con la clave `'products'`. Su tipo es `string | null`.
2. **Si es** `null`, todavía no se ha guardado nada (es la primera vez que se abre la aplicación en este navegador, o se ha borrado el `localStorage`): se devuelven los productos del mock, `PRODUCTS`. Es una cláusula de guarda, como en el paso 19.
3. **Si hay texto**, `JSON.parse(saved)` lo convierte de nuevo en un array. `JSON.parse` no sabe qué tipo de dato hay en el texto (para TypeScript devuelve `any`), así que con `as Product[]` se indica que es un array de productos.
4. `: Product[]` después de los paréntesis indica el **tipo que devuelve el método**. Así TypeScript comprueba que todos los `return` devuelven un array de productos.

Por qué cargar no se hace en el `effect`

Leer los productos guardados se hace **una sola vez**, al crear el signal, y no tiene que repetirse cuando `products` cambia. No es una reacción a un cambio, así que **no es trabajo para un effect**:

Tarea	Cuándo	Dónde
Cargar la lista	una vez, al crear el componente	valor inicial del signal (`loadProducts()`)
Guardar la lista	cada vez que cambia `products`	`effect`

El flujo completo

La primera vez que se abre la aplicación:

1. Se crea el signal `products` y se llama a `loadProducts()`.
2. `getItem('products')` devuelve `null`: no hay nada guardado.
3. Se devuelve `PRODUCTS`: el signal empieza con los 20 productos del mock.

4. El effect se ejecuta por primera vez y guarda esos 20 productos en `localStorage`.

Al borrar un producto:

5. `deleteProduct()` actualiza `products` (19 productos).
6. El effect se dispara y guarda los 19 en `localStorage`.

Al recargar la página:

7. Se crea el componente de nuevo y se llama a `loadProducts()`.
8. `getItem('products')` devuelve el texto con los 19 productos.
9. `JSON.parse` lo convierte en un array: el signal empieza con **19 productos**. Se ve **Productos (19 de 19)**: el borrado se ha conservado.
10. El effect se ejecuta su primera vez y vuelve a guardar los 19 (no cambia nada).

Comprobado también al crear: tras borrar el producto 1 y crear "Producto de prueba", al recargar sigue ahí, con `id` **21** (el último producto, el 20, más 1, como en el paso 13).

Volver a los productos del mock

Como los datos se conservan, el mock **ya no se vuelve a cargar** al recargar. Para volver a empezar desde los 20 productos originales hay que borrar la clave `products`:

- Desde las herramientas de desarrollo: **F12** → **Application** → **Local storage** → `http://localhost:4200` → seleccionar `products` → botón **Delete** (o tecla Supr).
- O desde la **consola** (F12 → **Console**):

```JS
localStorage.removeItem('products');
```

Y recargar la página: `getItem` devolverá `null` y se usará otra vez el mock.

7. Cuándo usar `effect` y cuándo no

`effect` es potente, pero **no debe ser la primera opción**. La documentación de Angular recomienda usarlo **poco**, solo para sincronizar los signals con algo **externo** que no se puede expresar de otra forma.

✅ Casos en los que se recomienda

Caso	Ejemplo
Guardar datos en el almacenamiento del navegador	guardar `products` en `localStorage` (este paso)
Registrar cambios (*logging*) o depurar	`console.log` cada vez que cambia un signal
Modificar el DOM a mano, cuando la plantilla no basta	poner el foco en un campo, dibujar en un `<canvas>`
Sincronizar con librerías externas	actualizar un gráfico o un mapa de otra librería

En todos ellos el effect **comunica los signals con el exterior**: algo que no es un signal ni la plantilla del componente.

❌ Casos en los que no se recomienda

1. Calcular un valor a partir de otros signals. Para eso está `computed`:

```ts
// ❌ MAL: usar un effect para mantener un valor derivado
totalProducts = signal(0);

constructor() {
  effect(() => {
    this.totalProducts.set(this.products().length);
  });
}
```

```ts
// ✅ BIEN: un valor derivado es un computed (paso 21)
totalProducts = computed(() => this.products().length);
```

La versión con effect **funciona**, pero es peor:

- `totalProducts` es **escribible**: cualquier método podría cambiarlo por error y quedaría desincronizado.
- Durante un momento tiene un **valor incorrecto** (`0`) hasta que el effect se ejecuta, porque el effect no se ejecuta en el mismo instante.
- Se hacen **dos pasos** (cambiar `products`, luego el effect cambia `totalProducts`) en vez de uno, y es más difícil seguir de dónde sale cada valor.

Regla: si el effect solo sirve para hacer `set` en otro signal, casi siempre debería ser un `computed`.

2. Reaccionar a una acción del usuario. Si algo debe ocurrir **al pulsar un botón**, se escribe en el método del botón, no en un effect:

```ts
// ❌ MAL: un effect pendiente de showForm para vaciar los campos
effect(() => {
  if (this.showForm()) {
    this.clearFields();
  }
});
```

```ts
// ✅ BIEN: el método del botón Nuevo ya hace lo que tiene que hacer (paso 17)
newProduct() {
  this.clearFields();
  // ...
  this.showForm.set(true);
}
```

El método deja claro **qué pasa y cuándo**; con el effect habría que buscar por todo el componente qué ocurre al cambiar `showForm`.

Cómo decidir

```
¿Quiero obtener un VALOR a partir de otros signals?
    └── sí ──► computed()

¿Quiero que pase algo cuando el usuario hace algo (clic, escribir...)?
    └── sí ──► un método llamado desde el evento

¿Quiero que pase algo FUERA de Angular cada vez que cambie un signal
(guardar, registrar, tocar el DOM o una librería)?
    └── sí ──► effect()
```

8. Persistencia sin base de datos: ventajas y límites

Con `localStorage` la aplicación ya **recuerda** los cambios, pero es importante entender hasta dónde llega:

✅ Ventajas	❌ Límites
No necesita servidor, base de datos ni internet	Los datos están **solo en ese navegador y ese ordenador**
Muy sencillo: dos métodos (`setItem` y `getItem`)	Otro usuario, otro navegador u otro dispositivo **no los ve**
Los datos sobreviven a recargar y cerrar el navegador	El usuario puede **borrarlos** (o el navegador, en modo incógnito al cerrar)
Respuesta inmediata (síncrono)	Unos **5 MB** como máximo, y solo texto
	No es seguro: no sirve para datos privados

En una aplicación real con varios usuarios, los productos estarían en una **base de datos** en un servidor, y Angular los pediría y enviaría a través de una API (con `HttpClient`). Aun así, `localStorage` se usa mucho en aplicaciones reales para cosas **propias de cada usuario y no críticas**: el tema claro u oscuro, el idioma, los últimos filtros usados, un carrito de la compra antes de iniciar sesión…

Un detalle: si alguien modificara a mano el texto guardado y dejara de ser un JSON válido, `JSON.parse` lanzaría un error y la aplicación no cargaría hasta borrar la clave. Se podría proteger con un `try...catch`, pero se deja así para que el ejemplo sea más sencillo.

Qué pasa al borrar un producto y recargar

1. Primera vez: `loadProducts()` no encuentra nada (`null`) y devuelve el mock. Se ven **Productos (20 de 20)**. El effect se ejecuta por primera vez y guarda los 20 en `localStorage`.
2. El usuario escribe en el filtro: `searchText` cambia, pero el effect **no** se ejecuta porque no lee `searchText`.
3. El usuario pulsa **Borrar** en un producto: `deleteProduct()` hace `products.update(...)`.
4. `products` avisa de que ha cambiado. Angular recalcula los `computed`, actualiza la pantalla y **ejecuta el effect**, porque `products` es su dependencia.
5. El effect convierte los 19 productos a texto con `JSON.stringify` y los guarda con `setItem`.

6. El usuario recarga la página (F5): la memoria se borra y el componente se crea de nuevo.
7. `loadProducts()` encuentra el texto guardado, lo convierte con `JSON.parse` y el signal empieza con **19 productos**: **Productos (19 de 19)**.

Resumen

- Los signals funcionan de **forma autónoma**: recalculan los `computed` y actualizan la plantilla solos, pero **no hacen nada más**. Para hacer algo cuando cambia un signal se usa `effect()`.
- Un **effect** ejecuta su función **una vez al principio** y **cada vez que cambia** algún signal que lee. Los signals que **lee dentro** de la función son los **únicos que lo disparan**; este solo lee `products`.
- Se define en un **contexto de inyección**: el **constructor** o la declaración de una propiedad. Fuera de ahí da el error **NG0203**. Se destruye solo junto con el componente.
- `computed` calcula valores; `effect` produce efectos secundarios. Se recomienda para sincronizar con el exterior (almacenamiento, logging, DOM, librerías) y **no** para calcular valores derivados ni para reaccionar a clics.
- `localStorage` guarda pares **clave–valor de texto** en el navegador, de forma **persistente** y **por origen**. Métodos: `setItem`, `getItem` (devuelve `null` si no existe), `removeItem`, `clear`, `key` y la propiedad `length`.
- Los arrays y objetos se guardan con `JSON.stringify` y se recuperan con `JSON.parse`; sin ellos se guardaría `'[object Object]'`.
- `loadProducts()` carga los productos guardados (o los del mock la primera vez) como valor inicial del signal; el **effect** los guarda tras cada cambio. Así se consigue **persistencia sin base de datos**, limitada a ese navegador.